What It Takes to Run a 125B LLM Fast on a 64 GB Mac, and How to Go Faster

How batch-invariant speculative checks, SSD-streamed experts and a certified lossy layer build on each other within a 45 GB memory line, and what each piece adds.

Kamen Levi
Independent researcher

9 November 2026

Abstract

We run a 125B-parameter mixture-of-experts model (Qwen3.8-Flash-Next: 48 layers, 512 routed experts per layer, 10 per token) on one MacBook Pro (M5 Pro, 18-core CPU, 20-core GPU, 64 GB, 2 TB SSD), with the routed experts streamed from the internal SSD at about 3.2 bits per weight and the process's lifetime peak at most 45 GB on the benchmark prompts (plus a 7.45 GB expert tier that macOS may reclaim; longer prompts go over it, section 6.1). The pieces are mostly published; this paper is about how they depend on each other and what each adds. Speculative decoding pays one pass of the large model for up to 16 drafted tokens, but it is exact only if that pass computes the same bits as one-token decoding. Batch-invariant kernels on the GPU and its matrix units make it so, and we audit it on every run (96 of 96 code runs, 24 of 24 chat and 24 of 24 math runs token-identical to one-token greedy decoding). Exactness then turns every later optimisation into a test of token equality that takes minutes: the levers of the streamed engine, such as per-layer expert pools and expert reads hidden behind the GPU's work, were each adopted that way. On top of the exact engine, a lossy layer lets drafted rows of code answers skip cold experts and run 3-bit parts, and ships only if it scores the same as the exact reference on the model's benchmarks.

The exact configuration decodes 101.4 tok/s on 12 HumanEval prompts, 70.4 on 12 MBPP, 77.9 on 12 math and 37.1 on 12 chat (IFEval) prompts, at a lifetime peak of 41.7-42.5 GB [K]. The lossy configuration we ship scores the same as the exact reference on the model card's IFBench, GPQA Diamond and LiveCodeBench v6 in both prompt orders, and decodes 164.0, 112.7, 78.9 and 45.2 tok/s on the same sets at 42.4-43.2 GB [K]. Our first token is slow, a median of 2.7-4.6 s on these short prompts: on the same machine and prompts the open engine Strata's takes 0.54-1.01 s, and Strata wins or ties every whole request of up to 256 tokens, although our lossy decode is 2.5-3.5x its decode on code [K]. None of the methods is specific to this model's layers; we tested one model on one machine. Section 12 describes each finding that, to our knowledge, has not been published before, with its evidence, what it gives and its limits, including one idea that did not pay off. We also report negative results, including three headline numbers we corrected or withdrew.

---

1. Introduction

1.1 The setting

- **The model.** Qwen3.8-Flash-Next (arXiv 2608.30320) is an open-weight mixture-of-experts model with a hybrid trunk: 36 linear-attention (Gated DeltaNet, "GDN") layers and 12 full-attention layers, four residual streams joined by low-rank hyper-connection (HC) mixers, one shared expert and 512 routed experts per layer with 10 routed per token, and a multi-token-prediction (MTP) head. It has about 125B parameters in total (section 4.1).
- **The machine.** One MacBook Pro, M5 Pro, 64 GB of unified memory, internal SSD. All numbers are for one user, greedy decoding.
- **The problem.** At MLX 4-bit the routed experts alone are 67.9 GB [K, BM:FINDINGS.md section 1], so they cannot all stay in memory. A token reads about 2.46 GB of dense weights and 1.33 GB of routed experts [K, same]. Decoding one token at a time pays a full pass over the dense trunk per token.
- **Speculative decoding** pays that pass once per check: a small drafter proposes a block of tokens and one pass of the large model verifies up to 16 of them.

- **Why exactness is not free.** A verify pass computes 16 positions in one batched call; one-token decoding computes each in a 1-row call. On stock kernels those are different programs with different rounding: in our first screen all 16 rows differed from one-token decoding at layer 0 (section 3). A greedy decoder then drifts at near-ties, and "speculative decoding is lossless" stops being true.

1.2 The idea: known pieces that need each other

The pieces build on published work; section 8 credits each one and says which part is ours, and the prior art is in Appendix I. Section 12 describes the findings of this work that, to our knowledge, have not been published before. What this paper adds is the order in which they depend on each other, and that they still work, exactly, when combined. Section 7 gives each step's measured effect, and says where a step was never measured alone.

1. **Exact checks.** Speculative decoding verifies up to 16 drafted tokens in one pass, but greedy decoding is unchanged only if that pass computes the same bits as 16 one-token steps. Batch-invariant kernels make it so (section 3; [KERNELS]).
2. **Optimisation by token equality.** With an exact reference, a change to the engine is accepted when every token equals the reference on a handful of prompts, in minutes, with no quality argument (section 3.3). Every lever of the 45 GB engine was adopted that way (section 4.4).
3. **A store that fits.** The routed experts (120.8B parameters) live on the SSD as aligned records with a 3-bit part and a 4th-bit sidecar; each layer keeps a pool of them in memory, and a token's two top-ranked experts read both parts while the other eight read the 3-bit part (sections 4.1-4.2). The check loop is what makes streaming affordable: a check pays the SSD reads for the union of its rows' experts once, and every token it keeps shares them.
4. **Hiding the reads.** Expert ids stay on the GPU, and resident experts compute while missed ones load (section 4.4). The experts' outputs are still summed in a fixed order, so both changes keep every token equal and were accepted as in step 2.
5. **A certified lossy layer.** On drafted rows of code answers the engine skips cold lower-ranked experts and runs the 3-bit part of the rest, and a few reads are made at lower precision everywhere (section 5). The check loop says which rows are drafted; the layer ships only if it scores the same as the exact reference of step 1 on the model's benchmarks.
6. **Prefill from the same store.** A short prompt can build its experts from the copies already in memory and read only what is missing (PF-D, shipped for chat); a long prompt can run each layer over the whole prompt, so that each expert is read about once (built, not shipped; section 4.5).

1.3 What carries to other models

None of the methods depends on this model's linear-attention or hyper-connection layers (section 8). The exact check needs batch-invariant kernels for the model's operations; the store, the loader and the lossy row rules need a mixture of experts with a drafter. The design is meant as an engine for such models in general, but we tested one model on one machine, and that is the limit of what this paper shows.

1.4 Contributions

1. **Exact speculative checks at 125B on Apple silicon (section 3; kernels in [KERNELS]).** A 16-row check that equals 16 one-token steps bit for bit, audited on every measured run, and a rule for changing the reference.
2. **An engine built by token equality (sections 4 and 7).** An SSD-streamed store, cache and loader within a 45 GB memory line, each lever accepted by token equality, and an account of what each piece adds on top of the others and which steps were never measured alone.
3. **A lossy configuration certified on the model's benchmarks (section 5; method in [EVAL]).** The same scores as the exact reference on HumanEval+, MBPP, math and IFEval, and, for the configuration we ship, on the model card's IFBench, GPQA Diamond and LiveCodeBench v6, in both prompt orders.
4. **Methods that carry to other MoE models (section 8), a short comparison with other engines (section 9), and negative results (section 10) and the path to more speed (section 11).**
5. **New findings (section 12).** Findings that, to our knowledge, have not been published before, each with its evidence, what it gives, what is measured and what is still a hypothesis, and how it could be improved.

1.5 What we do not claim

- **Not the fastest engine for this model.** Our first token is 2.7-7.0x slower than Strata's on the same machine, and engines with lower-bit files or on larger Macs report a faster decode or prefill (self-reported; section 9). Our decode leads on code and math, slightly on chat in the lossy mode, and by a wide margin only in the lossy mode on code.
- **Not new pieces.** Every lever in the measured configurations builds on published work, credited in section 8 and Appendix I; the findings that are, to our knowledge, new are in section 12. What we add is making them work together, exactly, at this size on this hardware, and measuring what each adds. To our knowledge no published engine for this model combines all of them; Strata and Sushi combine several (section 9).
- **Not exact to the original 4-bit weights.** "Exact" always names a reference (section 2); ours runs ranks 0-1 of each token's experts at 4 bits and ranks 2-9 at 3 bits.

1.6 The companion papers

[KERNELS] has the batch-invariant kernels and the MLX findings; [EVAL] the certification method and its pitfalls; [DRAFTER] the drafter work, including what failed; [RELEASE] the engine package (GPL-3). The 32, 24 and 12 GB lines are the subject of a companion paper ([SMALL-RAM]); no number in this paper depends on it. This paper states each companion's results in one sentence where it needs them.

2. Definitions and how we measure

- **Exact (to R).** The output tokens, and the logits row behind every kept token, are bit-identical to one-token greedy decoding under reference R, a fixed set of weights, kernels and call shapes. Changing a kernel changes rounding and makes a new reference, which is accepted once on task scores (section 3.4). Our 45 GB reference is NAX6-N.
- **Lossy, same scores on .** A configuration that changes which computation a row runs, accepted only after its pre-registered same-scores gate against its exact reference on the named known benchmarks, in two prompt orders. Never "same quality".
- **Lossy, gate open** (a gate is running or undecided) and **lossy, ungated** (no gate). Every configuration in this paper carries one of these labels.
- **Speed** comes as a triple for a named prompt set:
  - whole decode = generated tokens / decode wall time (prefill excluded, every check included);
  - whole request = generated tokens / (first token time + decode wall time);
  - first token = request start to the first generated token, including prefill and setup (median and range);
  - plus the lifetime peak memory footprint of the process (ri_lifetime_max_phys_footprint, which counts the GPU's buffers), in GB of `10^9` bytes; the 45 GB line is 41.9 GiB [K, ES:evalsuite/decode.py, footprint_peak_gb];
  - not counted in it: a 7.45 GB volatile expert tier (64 / 32 slots per base / sidecar pool), purgeable memory that macOS reclaims before it compresses or swaps; every run log behind Table 6.1 states it [K, e.g. BM:data/rel/rli_160.txt line 7].
- **Aggregation.** Pooled over prompts and passes (total tokens / total time). The engine's run reports quote a geometric mean over prompts ("geo"), 0-8 % higher on our sets; both are in Appendix E.
- **Workloads.** 12 HumanEval and 12 MBPP prompts (the same ids in every run), 12 IFEval prompts as our chat set and 12 math (MATH-500) prompts; greedy, at most 256 new tokens, EOS stop. HumanEval flatters: 67.8 % of the generated tokens on our 12 HumanEval prompts copy the prompt's own signature and docstring [K, F:lossy350-thinking-hour], which drafting accepts cheaply. We never quote HumanEval without MBPP or chat beside it.
- **The machine during runs.** Other jobs were stopped through a GPU lock ("quiet"); a run's report lists pauses for heat and drops any request that overlaps one.
- **Marks.** [K] measured, [P] priced (arithmetic or a CPU replay on measured data), [G] judgement. Every number names its source (a run record, report or result file); Appendix C maps the keys to paths.
- **The memory line.** "45 GB" means the run's lifetime peak is at most 45.0 GB. It is the engine's own footprint on a 64 GB machine, not a limit the operating system enforces.

3. Exactness (details in [KERNELS])

3.1 References

reference	what it is	status
REF2	batch-invariant kernel set: every matmul, HC read and attention call padded to 16 rows, routed experts on one 16-row tile kernel, a load-time self-test	adopted 28 Oct (T1210)
REF2b	REF2 + attention key buckets past 512 keys, attention classes below 128 keys, the commit phase and slot call	adopted 31 Oct (T1219)
REF2-T	REF2b on rank-tiered experts: ranks 0-1 at 4 bits, ranks 2-9 at 3 bits	1 Nov (T1214, T307)
NAX6-N	REF2-T with the routed experts on a matrix-unit kernel over records in the kernel's native layout	certified 5 Nov; the 45 GB exact reference (EV:RESULT_nax6_2026-11-05)

The first screen compared 16 one-token steps with one 16-row verify on identical caches, layer by layer: with the shipped kernels all 16 rows diverged at layer 0, because the framework picks a different matmul kernel by row count; attention, long context (past 512 keys) and short context (below 128 keys) broke next [K, LT:'R-40 REF2'; T1225; T1208; T1218]. The fixes and the kernels are [KERNELS]'s subject.

3.2 The audits

Each audit decodes every prompt twice, with the speculative loop and one token at a time, and compares every kept row.

reference and arm	prompts	result	source
REF2b	36 full-length (12 code, 12 math, 12 IFEval)	16,305 / 16,305 kept rows bitwise; 15 / 15 prompts past 512 keys, 6 / 6 past 1,024	EV:RESULT_r56b_ 2026-10-31
NAX6-N + out-of-order experts	same 36	15,690 / 15,690 kept rows bitwise; out-of-order path engaged on every tier call	EV:RESULT_nax6_ 2026-11-05, Part 2
the exact default (E, EB) on code	12 HumanEval + 12 MBPP, 2 arms x 2 passes	96 / 96 runs token-identical to one-token decoding	F:2026-11-07_1735
EB on chat and math	12 IFEval + 12 math, 2 passes	24 / 24 and 24 / 24	F:2026-11-07_2330
E + coherent reads (+ the 1-row check) on chat	12 IFEval	12 / 12 in two runs (37.8 / 37.9 tok/s) and 12 / 12 (38.3); without coherent reads 3 / 12	F:2026-11-08_0750
layer-major prefill	3 agent prompts, 1.6K-7.9K tokens (last chunks of 541, 634 and 764 tokens)	decode tokens equal; logits hash equal on 8 / 8, 28 / 28, 22 / 22 checks	F:2026-11-08_0800
layer-major prefill, short last chunk (32 GB exact configuration, 64 tokens)	5,235 and 5,420 tokens (last chunks of 115 and 300 tokens)	logits hash equal on 0 / 20 and 20 / 20 checks; with the 115-token chunk the tokens differ from the 36th	LP:data/pfa_tail_ 1111.md

- **The chat failure and its fix.** A one-call path for chat checks made exact chat nondeterministic (3 of 12 prompts equal). Coherent reads in the expert kernel restored 12 of 12 at no cost (F:2026-11-08_ 0750); [KERNELS] has the mechanism.
- **The audit is part of every run.** Each exact row of section 6 carries its own token-identity count. It is how two bugs were caught that speed numbers alone hide: a fast path that never engaged (T18) and a kernel that was silently bypassed (F:flat2, caveat).

3.3 What exactness buys

Once a reference exists, an optimisation is accepted by token equality on a handful of prompts, in minutes, with no quality argument. Every lever of section 4.4 was accepted this way. It has a price: the batch-invariant one-token path ran at x0.75 of stock kernels (11.65 vs 15.50 tok/s over 210 rows, row time with prefill included; information only) [K, EV:RESULT_ref2_2026-10-28; [KERNELS] R31].

3.4 Changing the reference: certified once, on scores

A new reference moves bits, so it is judged on tasks, once, by a rule fixed in advance ("decision 4"): on each of three sets (math, n 50; IFEval, n 100, strict prompt level; HumanEval+, n 60, pass@1) the paired score difference must be above -5 points and the exact McNemar p at least 0.05.

new reference vs old	math	IFEval	HumanEval+	verdict	source
REF2 vs the previous reference	+8.0	-3.0	0.0	PASS	EV:RESULT_ref2_2026-10-28
REF2b vs the previous reference	+6.0	-3.0	0.0	PASS	T1219
REF2-T vs REF2b	0.0	+3.0	+3.3	PASS	T307
NAX6 vs REF2-T	0.0	-3.0 (p 0.375)	-3.3 (p 0.50)	PASS	EV:RESULT_nax6_ 2026-11-05

These are same-score tests on 210 items, not proofs of equal quality: pooled over the NAX6 sets, NAX6 alone is right on 2 items and REF2-T alone on 7 (two-sided p 0.18). A further kernel bundle (REF2c-T) failed the rule on IFEval (-5.0) and was not adopted. The gaps chain: the certified lossy config of section 5 is -8.33 points on the HumanEval+ core set against REF2-T, two references back (forward order, p 0.0625), which no gate of its own measured [K, F:2026-11-08_0455; information].

4. The 45 GB system

4.1 Model, store and hardware

part	parameters	notes
routed experts	120.8 B	48 layers x 512 experts, top-10; one expert = gate/up [1280 x 2560] + down [2560 x 640]
linear attention (GDN)	2.09 B	36 of 48 layers
full attention + indexer	0.62 B	12 layers, 24 query heads, 2 key/value heads, head dim 256
hyper-connections	0.64 B	4 residual streams, low-rank mixers
embedding / output head / shared expert / router	0.64 / 0.64 / 0.24 / 0.06 B	one shared expert per layer
MTP head	2.61 B	one layer with its own 512 experts
n-gram tables	51.2 B	layer 1 only; 16 rows read per token, from the SSD

Source: BM:FINDINGS.md section 1 (config and safetensors headers). **Active per token [P, arithmetic on this table]:** 10 of 512 routed experts in each of 48 layers (480 x 4.92 M = 2.36 B), attention 2.71 B, hyper-connections 0.64 B, the shared expert and router 0.30 B: 6.0 B, or 6.65 B with the output head. The 125.7 B total counts the rows above except the MTP head, the vision tower and the n-gram tables (16 rows of 160 values read per token); with them the checkpoint stores 180 B, the size some engines quote for this model (Kapitsa, Mference; Appendix H). Our engine runs this architecture through the model code of MTPLX by Youssof Altoukhi (Apache-2.0; github.com/youssofal/mtplx); the store, the kernels and the check loop are ours.

- **Records.** Each expert is one contiguous, 16 KB-aligned record, read with one call that bypasses the page cache (F_NOCACHE): an MSB part (the 3 most significant bits of every weight with scales and biases) and an LSB sidecar (the 4th bit). The 4-bit weights are rebuilt bit-exactly from the two parts; the store is 68.5 GB on disk (48 layer files; 2.79 MB per expert record) [K, BM:models/bare/records3, the converter's output; not in the public snapshot]. For the matrix-unit kernel the records are stored in the kernel's own fragment order, with scales packed to 11 bits (bitwise; T527, T334).
- **Rank tiers.** A token's two highest-scoring experts (ranks 0-1) run at 4 bits and read both parts; ranks 2-9 run from the MSB part alone. Expert bytes fall about 20 % (T307).
- **The SSD.** Cold 2.77 MB reads reach 14.0 GB/s at queue depth 16 [K, BM:FINDINGS.md section 2]. In the engine the SSD time per check follows 10-16 ms plus 0.15-0.17 ms per missed expert [K, T288].

4.2 Expert cache and loader

- Each layer has a pool of expert slots in GPU memory; C 256 means 256 slots per layer on average, with per-layer sizes at equal memory (-17.2 % SSD bytes per kept token and +8.3 % tok/s, exact; T12, adapted from AdapMoE). Eviction is LRU.
- All misses of a layer load in parallel, aligned, straight into slot memory (T7; M-29). Part of the pool is a volatile tier the operating system may reclaim (T1088).
- **The wired limit.** MLX's wired-memory collector unwired the expert pools during prefill, and each prompt's first check re-wired about 31.8 GB, blocking for about 250 ms. One call after load (mx.set_ wired_limit, published practice in mlx-lm) removes it, bitwise in the exact mode: +25.1 % whole decode, pooled, on the lossy config (144.1 -> 180.1 tok/s, 12 HumanEval, two passes each; the notes

give geo means, 142.2 -> 181.2) [K, BM:data/r57/s6{b,w}{1,2}_256_runs.json; F:2026-11-07_0900, F:2026-11-07_0945].

4.3 The check loop

1. **Draft.** A block drafter (our fine-tune of DFlash2 by Inco AI, Apache-2.0; [DRAFTER], Appendix I) proposes up to 16 positions in one pass; chat uses the model's own MTP head, chosen by task (T782). Drafts are kept while the running product of the drafter's confidences stays at or above 0.3 (T309).
2. **Verify.** One padded 16-row pass of the 125B model checks the drafts; they are accepted up to the first mismatch, and the row after the last accepted draft gives one more token (the bonus row).
3. **Commit.** The recurrent and attention states advance by the kept rows only.

Tokens kept per check decide the speed and depend on the workload: 10.7 per check on HumanEval, 6.0-6.1 on MBPP, 6.5 on math and 2.5 on chat (kept tokens over checks, pooled) [K, F:2026-11-07_1735; F:2026-11-07_2355]. Chat checks are short because the drafter is unsure, and their cost is the low-row check's fixed cost (about 51 ms exact, 40 ms lossy at 1-2 rows, against a memory floor near 15-20 ms) [K, F:2026-11-07_2355].

4.4 The exact default and its levers

Every lever was adopted on "every token equal to the reference" plus a speed line written before the run. The first five rows give each lever's effect as measured then, in its own run's metric (often a geo mean or the steady rate, before the 7 Nov correction in section 10); they do not compare with Table 6.1.

lever	what it does	measured effect	source
sync-free expert gate	expert ids stay GPU data; the GPU waits on slots the loader publishes, no CPU-GPU sync per layer	warm check + commit 232.6 -> 117.3 ms; +35 %	T489, T476
confidence cut at 0.3	section 4.3	+19 %	T309
out-of-order experts	resident experts compute first, missed ones as their reads land, in a fixed bitwise order	+13 to +18.7 %	T14, T490
per-layer pool sizes	section 4.2	+8.3 %	M-17
drafter by task	MTP head on chat	chat +24.5 % (time-weighted) on 19 fresh chat prompts	T782
SSD keepalive	section 8.1	about -3 ms per check	BM:docs/NIGHT_1104.md:37
matrix-unit experts on native records (NAX6-N)	section 3.1	+6.2 % whole decode, pooled (44.74 vs 42.12 tok/s, 13 prompts, C 192; +6.7 % on the steady rate, information only)	LP:data/natfri/natfri_pooled.txt; LL, 'NATIVE-FRI run 2'
cache keep	the loop no longer clears the kernels' per-weight caches before every check	-7.2 to -8.0 ms per check; x1.06 whole decode on the lossy config, pooled (x1.10 on the steady rate, information only); tokens equal	F:2026-11-07_0144; BM:data/r57/sc{b,k}{1,2}_256_runs.json
wired limit	section 4.2	removes a ~250 ms first-check stall per prompt	F:2026-11-07_0900
the exact-safe bundle (EB)	a bundle of kernel changes (seams a-e, DQ1; [KERNELS]), bitwise	+3.7 % HumanEval, +2.9 % MBPP (pooled)	F:2026-11-07_1735; Appendix E
coherent reads	GPU reads of loader-written memory made coherent	exact chat 3 / 12 -> 12 / 12, no cost	F:2026-11-08_0750
the 1-row check	a 1-row form of the check	bitwise (676 / 676 logits hashes); +1 % exact chat, +4.2 % lossy chat	F:2026-11-08_0750

4.5 Prefill

Today's prefill runs the prompt in 1,024-row chunks and streams each chunk's experts from the SSD, so a long prompt reads the expert store several times (430 GB for a 7.9K-token prompt). Layer-major prefill runs each layer over the whole prompt and reads each expert about once (68 GB): 1.9-3.4x faster at 1.6K-7.9K tokens, and exact except when the prompt's last 1,024-token chunk is under about 200 tokens; it runs only on prompts over 1,024 tokens (section 6.2) [K, F:2026-11-08_0800; LP:data/pfa_tail_1111.md]. The package's terminal chat (chat.py, package 1.1.0) runs it by default at 32 and 24 GB; at 45 GB it is not shipped ([RELEASE] section 4). On short prompts the first token is still 2.7-4.6 s (section 6.3), except on chat with PF-D.

PF-D builds a short prompt's routed experts from the copies already in memory and reads only the missing parts from the SSD, instead of re-reading every routed expert (20.4 -> 7.5-9.6 GB per chat prompt [K]); its version 4 also reads the next layer's missing experts during the current layer. Both follow Strata's prompt-path design (read only what memory lacks; read the next layer's misses during this layer); no Strata code is included ([RELEASE] section 7). PF-D is exact: on the shipped chat configuration its tokens (12 / 12) and logits hashes (721 / 721) equal the same configuration without it, and a check arm on the exact configuration (16 tokens per prompt) found all 87,912 staged rows equal to the reference rebuild and its tokens equal to one-token decoding on 12 / 12 prompts [K, BM:data/pfd/pfd_report_0009.txt]. It stays exact on 10K- and 30K-token prompts, where it runs once per layer and 1,024-token chunk: tokens and logits hashes (25 / 25, 27 / 27) equal the same configuration without it, and a check arm found all 705,420 staged rows equal to the reference rebuild. There it saves only 3-4 % of the first token and adds about 2 GB of lifetime peak [K, BM:data/pfd_long/summary.txt]. It ships on for chat at 45 GB only (Table 6.1c); every other row of section 6 was measured without it, and it was not run at 32 GB.

5. The lossy configuration and its certificate (method in [EVAL])

5.1 What it changes

The certified configuration (snapshot d8eb2e4; its full switch list is `BM:data/stack/cert_env.txt`) keeps the check loop and changes what some rows compute. Grouped by effect:

- **Lossy on code sets only (HumanEval, MBPP):** QOM-KEEP: on a drafted row, ranks 0-1 always run; a rank 2-9 expert runs only if the check loads it anyway or it is already resident, and otherwise drops out at weight 0, with no rerouting; the kept experts of drafted rows run from the 3-bit part (U3). Section 8.2 has the evidence.
- **Lossy everywhere (bits change on every set):** 8-bit hyper-connection mixers read in one pass on the matrix units (HC8); a split of K for narrow dense matrix-vector calls (DN-KS, rounding-level); single-call attention; small weights in the native layout; the matrix-unit expert form of the above. Single-call attention is bitwise below 512 keys.
- **Acceptance:** exact everywhere, except inside '#' comments of code answers, where a draft ranked first or second with at least a quarter of the top token's probability is kept (section 8.3).
- **Drafting:** one extra draft position so that the bonus row is gated too (DRAFT17); 17-32-row drafter projections on the matrix units; a drafter head over a reduced vocabulary; a copy check that continues a verbatim copy of the prompt in the draft (2,888 copied tokens accepted on 12 HumanEval prompts; F:2026-11-07_0025). It only drafts: the longest 8-16-token match in the committed context replaces drafts while a per-session duel says the copy wins at least 60 % of its disagreements with the drafter; the check still decides every token; a row whose draft the copy changed never counts as sure for the lossy row rules; it is off on math [K, code read: BM:scripts/r57_screen.py, R57_COPY=g2].
- **Exact plumbing:** lazy GDN commit, submitting work every two layers, the cache keep and the wired limit.

5.2 The certificate [K]

Against NAX6-N, forward and reverse prompt order (EV:RESULT_cert, 2cfed64; results/cert/CERT_VERDICT.txt):

set	items	forward	reverse
HumanEval+	164	90.2 vs 92.7 % (-2.44, p 0.125)	92.7 vs 92.7 % (0.00)
math	100	86.0 vs 86.0 %	84.0 vs 86.0 % (-2.00, p 0.625)
MBPP	100	76.0 vs 79.0 % (-3.00, p 0.375)	81.0 vs 79.0 % (+2.00)
IFEval (chat)	100	86.0 vs 87.0 % (-1.00)	85.0 vs 87.0 % (-2.00, p 0.754)
pooled: HumanEval+, math, MBPP (728 pairs; IFEval not pooled)		-0.96 points, p 0.265: PASS (bar: above -2 points and p >= 0.05)	
long code answers	41 rows, 55,966 tokens	0 blunders: PASS	

- Every bar was registered before any row ran (PREREG_cert and amendments 2-3).
- **What it does not cover.** Multilingual benchmarks were not part of it; the claim is "same scores on these four English sets". It covers this configuration (CERT) only. L, which adds the exact-safe bundle

(Table 6.1c), was never gated by itself; the configuration we ship, L + coherent reads, passed its own gate on the model card's benchmarks on 9 Nov (Table 6.1b).

- **Chat is lossy in this configuration,** although chat uses exact acceptance: only 1 of 12 IFEval rows (CERT; L: 6 of 24, N45) equal one-token decoding, because the "lossy everywhere" parts change bits on every set [K, F:2026-11-08_0455; F:2026-11-07_2330]. A bisect on chat, one part at a time on the exact base, found two besides the 8-bit hyper-connection read (lossy by design): the one-call expert path (3 and 4 of 12 equal; nondeterministic until coherent reads) and the native layout for small weights (3 of 12); every other part kept 12 of 12 [K, BM:data/chs/chs3_report.txt].

5.3 The chat lever and its card gate

M1-B extends QOM's keep rule to the MTP head's chat drafts. With coherent reads it decodes 49.8 tok/s on chat, +13 % over L's chat runs (Table 6.1c; 44.2 tok/s in CHS4, an earlier run; expert misses per check about 92 -> 25) [K, Appendix E; BM:data/chs/chs4_report.txt; F:2026-11-08_0750]. Its IFEval gate passed (86 vs 87 forward, 83 vs 87 reverse; EV:RESULT_chat_m1b_2026-11-08). On a multilingual chat set it produced 334 low-probability tokens per 10,000 (a token-level line, information only) [K, same]. Its release gate is the card gate: the model card's benchmarks (IFBench, GPQA Diamond, LiveCodeBench v6) against the exact reference in both orders. It passed on 9 November (card arm m1b, 6 of 6 lines): M1-B + COH is "lossy, same scores on IFEval, IFBench, GPQA Diamond and LiveCodeBench v6 (both orders)" [K, EV:RESULT_card_m1b_2026-11-09; [EVAL] section 4.5].

---

6. Results

6.1 The 45 GB line: whole decode, whole request, first token

Setup for every row: 12 prompts per set (HumanEval and MBPP: the S5 ids; IFEval, our chat set, and math (MATH-500): the N45 ids), greedy, at most 256 new tokens, EOS stop, expert cache C 256, quiet machine (GPU lock), two passes unless noted. Whole decode and whole request are pooled over prompts and passes; first token is the median over prompts and passes (min-max) and includes prefill and setup. Every row is re-derived from the raw run records (`BM:data/r57/<tag>_256_runs.json`) by `MS:papers/main_table.py`; Appendix E lists the run reports' geometric means beside it.

Table 6.1a. EXACT: token-identical to one-token greedy decoding under NAX6-N [K].

set	config (run, tags)	whole decode tok/s	whole request tok/s	first token s	lifetime peak GB	runs == one-token decoding	source
12 HumanEval	EB = exact default + the exact-safe bundle (E5; e5hb1-2)	101.4	32.6	4.41 (3.85-6.23)	42.2 (per set)	24 / 24	F:2026-11-07_1735; BM:data/r57/ e5hb{1,2}
12 HumanEval	E = exact default (E5; e5h1-2)	97.8	31.7	4.59 (3.72-6.26)	42.2 (per set)	24 / 24	same; e5h{1,2}
12 MBPP	EB (E5; e5mb1-2)	70.4	21.7	3.80 (3.32-4.85)	41.9 (per set)	24 / 24	same; e5mb{1,2}
12 MBPP	E (E5; e5m1-2)	68.4	21.8	3.73 (3.26-5.60)	41.9 (per set)	24 / 24	same; e5m{1,2}
12 IFEval (chat)	EB (N45; nie1-2)	37.1	20.8	2.89 (2.23-3.88)	41.7	24 / 24	F:2026-11-07_2330; BM:data/n45/n45_ report.txt
12 IFEval (chat)	E + coherent reads + 1-row check (CHS4; yes1, one pass)	38.3	21.3	2.82 (2.34-3.71)	41.7	12 / 12	F:2026-11-08_0750; BM:data/chs/chs4_ report.txt
12 math	EB (N45; nme1-2)	77.9	38.9	2.73 (2.52-6.75)	42.5	24 / 24	F:2026-11-07_2330

- E = NAX6-N kernels + the exact levers (section 4) + cache keep + the wired limit; EB adds a bundle of kernel changes that are bitwise in this mode. "Exact" is to NAX6-N, whose experts run ranks 0-1 at 4 bits and ranks 2-9 at 3 bits.
- E5 prints the lifetime peak per set (both arms together), not per arm.
- Exact chat needed one fix: without coherent reads the one-call path matched one-token decoding on 3 of 12 prompts; with them 12 / 12 in two runs, at no cost (37.8 / 37.9 tok/s) [K, F:2026-11-08_0750].

S5's run records carry no first-token field, so Table 6.1b takes whole request and first token for the certified flags from S5' (same flags, a later snapshot, same sets).

The three lossy configurations at 45 GB. CERT is the certified configuration "v1' + wired" (BM:data/stack/cert_env.txt; section 5). L adds the exact-safe bundle, six switches that are bitwise in the exact mode (BM:data/stack/s5p_env.txt). L + COH adds coherent reads (BM:data/stack/lcoh_env.txt); it is the package's lossy configuration ([RELEASE] section 2). A gate covers only the configuration it tested, so each carries its own label.

Table 6.1b. LOSSY with a passed gate [K]: the configuration we ship (L + COH) and the certified one (CERT).

set	config, run (tags)	whole decode	whole request	first token s	lifetime peak GB	gate	source
12 HumanEval	L + COH as shipped, snapshot rel1 (rlh, one pass)	164.0	36.2	4.60 (3.78-6.07)	42.9	same scores as NAX6-N on IFBench, GPQA Diamond and LiveCodeBench v6, both orders (card arm lcoh)	BM:data/rel/rel_rows_report.txt (53dfff5); BM:data/r57/rlh_256_runs.json
12 MBPP	L + COH (rlm, one pass)	112.7	25.3	3.72 (3.35-4.88)	42.6	same	same; rlm
12 IFEval (chat)	L + COH (rli, rlr: two processes)	45.2	22.4	3.01 (2.41-3.81)	42.4	same	same; rli, rlr
12 math	L + COH (rlt, one pass)	78.9	37.9	3.20 (2.71-6.80)	43.2	same	same; rlt
12 HumanEval	CERT: S5, the certified build d8eb2e4 (s5v1-2)	175.3	not recorded (see S5')	not recorded	42.9	same scores as NAX6-N on HumanEval+, MBPP, math and IFEval, both orders (the certificate)	F:2026-11-07_1430; BM:data/r57/s5v{1,2}
12 HumanEval	CERT: S5', same flags, snapshot 6db4c24 (spb1-2)	171.0	36.0	4.56 (4.01-6.40)	42.9	same	BM:data/s5p/s5p_report.txt; spb{1,2}
12 MBPP	CERT: S5 (s5mv1-2)	111.1	not recorded (see S5')	not recorded	42.6	same	F:2026-11-07_1430
12 MBPP	CERT: S5' (spmb1-2)	114.5	25.8	3.79 (3.43-4.95)	42.6	same	BM:data/s5p/s5p_report.txt

- **The card gate (L + COH)** [K, EV:RESULT_card_lcoh_2026-11-09]: the model card's IFBench (150 keys), GPQA Diamond (100) and LiveCodeBench v6 (131) against NAX6-N in forward and reverse order: deltas -3.33 / -3.33, +3.00 / +3.00 and -1.53 / -3.05 points, exact McNemar p 0.29-0.69; the rule is the certificate's ([EVAL] section 4.5). These are not the four sets of the speed rows.
- **The certificate (CERT)** (details in [EVAL]): HumanEval+ (164), math (100), MBPP (100) and IFEval (100), forward and reverse order, each set within -5 points with exact McNemar p >= 0.05, and HumanEval+, math and MBPP pooled over 728 pairs -0.96 points (p 0.265); 0 blunders in 41 long code answers (55,966 tokens) [K, EV:RESULT_cert (2cfed64); results/cert/CERT_VERDICT.txt; F:2026-11-08_0455]. Multilingual known benchmarks were not part of it.
- **CERT's speed rows are code only.** The IFEval and math rows this table carried until 9 Nov (N45), and the chat runs we compared M1-B with (CHS4), ran L, not CERT [K, BM:scripts/n45.sh l30-31; BM:scripts/chsb.sh l34]; they are in Table 6.1c now.
- **L + COH against L** (Table 6.1c; other runs, not an A/B): whole decode -9.5 % on HumanEval, -5.9 % on MBPP and -9.2 % on math, +4.1 % on IFEval; the cause was not measured [K]. Coherent reads fix the one-call path's stale-slot reads (Table 6.1a's note); without them, chat outputs differ from run to run [K, F:2026-11-08_0750].
- Lossy runs need not repeat token for token: the two S5' passes (CERT) give the same tokens on 8 of 12 HumanEval prompts, because several lossy parts depend on the cache state [K, spb1 / spb2 token files]; this is why the gates use two prompt orders. L + COH repeated its IFEval run token for token (12 / 12, separate processes) and equals one-token decoding on 5 / 12 HumanEval, 5 / 12 MBPP, 2 / 12 IFEval and 5 / 12 math prompts [K, BM:data/rel/rel_rows_report.txt]. We call both "lossy, same scores", never exact.

Table 6.1c. LOSSY: L by itself (not gated) and the chat configurations (status per row) [K].

set	config (run, tags)	whole decode	whole request	first token s	lifetime peak GB	status	source
12 HumanEval	L = CERT + the exact-safe bundle (S5'; spx1-2)	181.3	36.6	4.51 (3.96-6.25)	42.9	not gated by itself: the certificate covers CERT and the card gate L + COH; the bundle is bitwise in the exact mode	BM:data/s5p/s5p_report.txt
12 MBPP	L (spmx1-2)	119.8	25.3	3.76 (3.37-5.40)	42.6	same	same
12 IFEval (chat)	L (N45; nil1-2)	43.4	21.1	3.00 (2.43-3.93)	42.4	same	F:2026-11-07_2330; BM:scripts/n45.sh l30-31
12 math	L (N45; nml1-2)	86.9	39.6	2.96 (2.71-6.98)	43.1	same	same
12 IFEval (chat)	M1-B + coherent reads (CHS5; z0, z0b)	49.8	24.4	2.92 (2.34-3.81)	42.4	same scores on IFEval (86 vs 87 forward, 83 vs 87 reverse) and on the card's IFBench, GPQA Diamond and LCB v6, both orders (arm m1b, EV:RESULT_card_m1b_2026-11-09)	F:2026-11-08_0750; EV:RESULT_chat_m1b_2026-11-08; BM:data/chs/chat_ttft.txt
12 IFEval (chat)	+ the 1-row check (CHS5; zs, one pass)	51.9	24.8	2.98 (2.51-3.65)	42.4	as M1-B + COH; its tokens equal the first chat row's (z0) on 12 / 12	BM:data/chs/chs5_report.txt; BM:data/r57/{zs,z0}_256_tokens.json
12 IFEval (chat)	chat as shipped: zs + PF-D v4 + read-ahead (section 4.5; pdz4a, one pass)	50.7	28.8	2.26 (1.55-2.59)	43.7	as zs (M1-B + COH); PF-D is exact: tokens 12 / 12 and logits hashes 721 / 721 equal its control without PF-D (pdz0, same prompts: 50.3 / 24.6 / 2.91 (2.34-3.74), 42.4 GB)	BM:data/pfd/pfd_report_0009.txt (snapshot 36ee120); pdz0: BM:data/pfd/pfd_report_2355.txt (366c452); records BM:data/r57/pdz{4a,0}_256_runs.json
12 IFEval (chat)	+ bundle / HC8_G + rank cut 8 + R1 (CHS5; zsgcr, one pass)	56.0	24.6	2.94 (2.55-3.74)	42.4	ungated	same

- L is lossy on chat and math too: 6 of 24 IFEval and 7 of 24 math runs equal one-token decoding, although those sets use exact acceptance [K, F:2026-11-07_2330].
- The PF-D rows are one pass each, from two runs a quarter of an hour apart. Without the read-ahead (PF-D v3) the median first token was 2.25-2.41 s in three arms over the two runs, so the read-ahead's own share is within the run-to-run spread; it reads more (9.6 against 7.5 GB per prompt) because 19-31 % of what it stages goes unused [K, same reports]. A lossy form (experts whose 4th-bit part is not in memory run at 3 bits) reaches 1.83 s and 30.5 tok/s whole request but changes tokens on 10 of 12 prompts; it is ungated and ships off [K, pdzfa in the same report].

Long prompts go over the memory line [K]. On three long agent prompts (1.6K-7.9K tokens) the certified lossy config peaks at 45.1 GB and the exact config at 44.5 GB (F:2026-11-07_2330, FLAG 2); the layer-major prefill of section 6.2 peaks at 45.10 GB (F:2026-11-08_0800). Both need a trim (a smaller C or an earlier scratch release) before they count at 45 GB. No trimmed run was made before the release: on such prompts the engine exceeds the 45 GB line by up to 0.1 GB [K]. Prompts of 10K and 30K tokens go further over it: the chat configuration as shipped peaks at 47.5 and 48.6 GB on them (45.2 and 46.7 GB with PF-D off) [K, BM:data/pfd_long/summary.txt], and the 32 GB chat configuration at 34.0 and 35.5 GB [K, LP:data/prefill_1109.md]. Every row of Tables 6.1a-c is at or under 43.7 GB (the PF-D row; every other row at or under 43.2 GB).

6.2 Prefill on long prompts

prompt (agent set)	today's prefill, 1,024-row chunks: first token (prefill tok/s)	layer-major prefill: first token (tok/s)	SSD read per prompt
1,565 tokens	15.9 s (98) [N45]; 15.2 s (103) [PFX]	8.0 s (195)	109 -> 64 GB
4,730 tokens	40.1 s (118); 37.3 s (127)	12.9 s (368)	265 -> 67 GB
7,932 tokens	64.1 s (124); 61.9 s (128)	18.0 s (441)	430 -> 68 GB

- Layer-major prefill runs each layer over the whole prompt and reads each expert about once; its decode

tokens and logits hashes equal today's on every check (8 / 8, 28 / 28, 22 / 22) [K, F:2026-11-08_0800; BM:data/pfx/pfx_report.txt]. Peak: 45.10 GB (see above). These prompts' last 1,024-token chunks are 541, 634 and 764 tokens long [K, same report].

- It runs only on prompts over 1,024 tokens (shorter ones take the default path), and it is exact only when the prompt's last 1,024-token chunk is long enough. On the 32 GB exact configuration, a prompt of 5,235 tokens (5 x 1,024 + 115) gives logits hashes equal to the default prefill's on 0 of 20 checks, and its tokens differ from the 36th; a prompt of 5,420 tokens (a 300-token last chunk) gives 20 of 20 [K, LP:data/pfa_tail_1111.md]. Runs of the same two paths on the 32 GB chat configuration put the switch between a last chunk of 197 tokens (differs) and one of 208 (equal) [K, LP:data/agent_cc_1111.md]. The likely cause [G]: on a short call MLX picks another gather kernel for the layer-major prefill's full expert scratch (all 512 experts' rows) than for the default prefill's compact one (the call's own experts); both paths are deterministic but round differently. The other prompts on which this paper and [KERNELS] compare with the layer-major prefill end in chunks of 292 tokens or more, so those results stand. The 24 and 45 GB lines were not checked on a short last chunk; the mechanism is the same there [G].
- The certified lossy config prefills like the exact one (16.3 / 41.2 / 66.6 s) [K, N45].
- Larger chunks alone (2,048-8,192 rows) change the logits and are out (section 10). A faster prefill form (PF-B: one MoE call per layer, with read-ahead) is exact on the three agent prompts (1.6K-7.9K tokens) since its gate fix: tokens and logits hashes equal the layer-major prefill's (8 / 8, 28 / 28, 22 / 22) at x1.12-1.21 its speed (500 tok/s at 7.9K), but it peaks at 45.8-46.2 GB, over the line, so it does not ship [K, BM:data/pfd/pfd.log part B; [KERNELS] section 5.2]. It is not exact on a 30K-token prompt: on the 32 GB exact configuration its logits hashes equal the layer-major prefill's on 0 of 25 checks, both at 8,192 rows per call (the published setting) and at 4,096, and its tokens differ from the 40th on, while the layer-major prefill's hashes equal the default prefill's on 26 of 26 (its last chunk, 292 tokens, is long enough for it) [K, LP:data/prefill_1109/exact30k/summary.txt]. The cause is the prompt's last chunk, not its length: in PF-B's grouped call the router and the shared expert compute the rows of a short last chunk (292 tokens at 30K) with other bits than a per-chunk call does. Last chunks of 541 tokens or more are unaffected, and so is a 427-token last chunk alone in its call; prompts of 10K, 15K, 20K and 25K tokens are exact (hashes 25 / 25, 28 / 28, 25 / 25, 24 / 24) [K, LP:data/tue_afternoon_1110.md, sections 3 and 5]. Running those two parts per chunk slice fixes both failing prompts: 20 / 20 hashes on a 5,412-token prompt with a 292-token last chunk (0 / 20 without the fix) and 26 / 26 at 30K, at x1.15-1.17 the layer-major prefill's speed (555 tok/s at 30K) and 2.3 GB more peak there [K, same file, section 6]. The fix is in a research copy, not in the package. Its exactness rests on those two prompts and on the lengths that passed without it, which were not re-run with it.

6.3 Where a request's time goes

- On the four 12-prompt sets the first token takes 43-80 % of a whole request's wall time [P on K: Table 6.1, e.g. HumanEval lossy as shipped 218 tokens per task at 164.0 tok/s = 1.33 s of decode against 6.03 s per request; exact chat 3.79 of 6.77 s is decode]. Decode is no longer the bottleneck at these answer lengths; the first token is.
- Why it is slow on short prompts: a chat prompt's prefill reads about 20 GB of experts, because every prompt token routes 10 experts in each of 48 layers, and it re-reads experts that are already resident; first token = 0.32 s + GB / 7.33 GB/s [P: a least-squares fit on the 12 CT1 rows of BM:data/pfb/pfb_report.txt; section 11].
- PF-D (section 4.5) cuts the shipped chat configuration's read from 20.4 to 9.6 GB per prompt and its first token from 2.91 to 2.26 s; whole request rises from 24.6 to 28.8 tok/s and decode is unchanged (50.3 / 50.7) [K, Table 6.1c]. The fit above gives 1.63 s for 9.6 GB, so with PF-D the reads alone no longer set the first token [P on K].
- What it costs a user is section 9: against an engine with a 0.5-1.0 s first token, a decode up to 3.5x faster (on code) does not pay back within 256 tokens.

7. How the pieces build on each other

Steps 1-5 are in the order in which each needs the ones before it; steps 6 and 7 apply to any configuration. Each effect was measured when its piece was adopted, on the engine of that day and in that run's own metric (section 4.4), so the effects do not add up to Table 6.1, and we give no waterfall. No run removes one piece at a time from the final configuration.

step	piece	what it does	what later steps need from it	measured effect	source
1	batch-invariant kernels (REF2 to NAX6-N; section 3)	a 16-row check computes the same bits as 16 one-token steps	a reference against which any later change is a token-equality test	exactness, audited on every run (section 3.2); the one-token path ran at x0.75 of stock kernels (information only)	EV:RESULT__ref2__ 2026-10-28; EV:RESULT__nax6__ 2026-11-05
2	the check loop: a block drafter cut at confidence 0.3, the MTP head on chat (section 4.3)	one pass of the 125B model keeps 2.5-10.7 tokens per check, by task	a pass whose dense and SSD costs every kept token shares; drafted rows that step 5 can treat differently	confidence cut +19 %; MTP head on chat +24.5 % (time-weighted, 19 chat prompts); no A/B against one-token decoding on the final engine	T309; T782; F:2026-11-07__1735; F:2026-11-07__2355
3	the streamed store: aligned records, per-layer pools, rank tiers (sections 4.1-4.2)	the experts fit a 45 GB line; a check reads each missing expert once for all its rows	a loader whose reads can overlap the GPU's work	per-layer pool sizes -17.2 % SSD bytes per kept token, +8.3 % tok/s, exact; rank tiers about -20 % expert bytes (a new reference, certified on scores)	T12; T307
4	hiding the reads: the sync-free expert gate, out-of-order experts in a fixed order (section 4.4)	the GPU computes resident experts while missed ones land, with no CPU-GPU sync per layer	a loop whose per-check fixed costs then show (step 6)	warm check + commit 232.6 -> 117.3 ms (+35 %); out-of-order +13 to +18.7 %; both exact	T489, T476; T14, T490
5	the lossy layer (CERT; section 5)	drafted rows of code answers skip cold lower-ranked experts and run 3-bit parts; some reads at lower precision everywhere	-	CERT 171-175 tok/s on HumanEval and 111-114 on MBPP, against 101.4 and 70.4 for the exact default; the two differ in more than the lossy parts (section 5.1), so this is not an A/B of the layer	Tables 6.1a and 6.1b; Appendix E
6	per-check fixed costs: cache keep, wired limit, matrix-unit experts, the exact-safe bundle, coherent reads, the 1-row check (section 4.4)	removes per-check clears and a ~250 ms first-check stall; faster kernels; coherent reads make exact chat deterministic	-	cache keep -7.2 to -8.0 ms per check; wired limit +25.1 % whole decode (lossy configuration); matrix-unit experts on native records (NAX6-N, later the reference) +6.2 %; the bundle +3.7 % and +2.9 % on code; coherent reads 3 / 12 -> 12 / 12 exact chat prompts; the 1-row check +1 % exact chat, +4.2 % lossy chat	section 4.4
7	prefill from the same store (section 4.5)	PF-D builds a short prompt's experts from the copies in memory; layer-major prefill reads each expert about once	-	PF-D: chat as shipped, first token 2.91 -> 2.26 s, exact; layer-major 1.9-3.4x at 1.6K-7.9K tokens, exact unless the last chunk is under about 200 tokens (chat.py's default at 32 and 24 GB in package 1.1.0; not shipped at 45 GB)	BM:data/pfd/pfd__ report__0009.txt; F:2026-11-08__0800; LP:release/src/chat.py

- **Why exactness comes first.** The levers of steps 2-4 and 6 were each accepted by token equality, in minutes; rank tiers, which change bits on purpose, were certified once as a new reference. Without the reference of step 1, each lever would have needed a benchmark gate, as the lossy layer does (sections 3.3-3.4).
- **Why the check loop makes the store affordable.** A check's SSD time follows 10-16 ms plus 0.15-0.17 ms per missed expert [K, T288], paid once per check and shared by the tokens it keeps: 10.7 per check on HumanEval, 6.0-6.1 on MBPP, 6.5 on math and 2.5 on chat [K, F:2026-11-07__1735; F:2026-11-07__2355]. Chat keeps few tokens per check, so its cost is the check's fixed cost (about 51 ms exact at 1-2 rows) [K, F:2026-11-07__2355], which is why the per-check costs of step 6 matter most on

chat.

- **Why the lossy layer needs the others.** It decides per drafted row (step 2), reads the 3-bit part that the store already keeps separately (step 3), and is certified against the exact reference (step 1). It pays most on code, where its expert rules apply: the lossy configuration we ship decodes 164.0 tok/s on HumanEval and 112.7 on MBPP against 101.4 and 70.4 for the exact default, 78.9 against 77.9 on math, where those rules are off, and 45.2 against 37.1 on chat [K, Tables 6.1a and 6.1b].
- **Where pieces do not compose.** Exact pieces compose by construction, and the combination is audited on every run (section 3.2). Lossy pieces need not: on chat, relaxed acceptance alone scored IFEval 87 and 8-bit hyper-connection mixers alone 89, both together 84, against 90 for the reference (section 10.6). This is why the lossy layer is gated as one configuration, not part by part.

8. Methods that carry to other models

The engine is not tied to this model. We tested one model, on one machine, so whether the methods below carry is a property of the design, not a measurement.

The second column names the published work each method builds on, with our part where it matters. "Carries": MD = any MoE with a drafter, SD = any speculative-decoding LLM, MAC = Apple's SSD or Macs, AGPU = MoE on Apple's GPU. None of these is specific to this model's GDN or HC layers (INV).

method	builds on	status here	carries	evidence
batch-invariant 16-row kernel set + key buckets	batch invariance (He and Thinking Machines Lab, 2025); ours: the 16-row form on Apple's GPU and matrix units, with key buckets	live, exact	SD, AGPU	section 3; [KERNELS]
SSD wake penalty + keepalive	NVMe idle power states; keeping drives awake with periodic I/O; ours: the penalty on Apple's internal SSD at sub-millisecond gaps and the read spacing that removes it (section 12.1)	live, exact	MAC	section 8.1
wake-ahead ping	waking a device ahead of predicted I/O (HotOS '15, "Software Engagement with Sleeping CPUs"); ours: one 4 KB read about 200 us before a burst, timed on this SSD	passed in microbenchmarks, not live	MAC	T47, T98
aligned zero-copy reads into GPU slots	direct I/O into device-visible buffers (F_NOCACHE / O_DIRECT; GPUDirect Storage); ours: the measurements into Metal slots on this SSD	live in the store	MAC	NOV, M-29
loader-to-GPU handoff by a coherent spin	MLX's polling-based fast CPU-GPU synchronization fence (MLX pull request #1773, 2025); ours: fresh lines per pass, per-layer events and residency published a pass ahead, between the SSD loader and the expert kernel	live, exact	MAC, AGPU	T76
OS-reclaimable (volatile) expert tier	macOS purgeable memory; ours: holding part of the expert pool in it	live, exact	MAC	T1088
rank-tiered records (4 bits for ranks 0-1, 3 for 2-9)	mixed precision across experts (HOBBIT; MC-MoE); ours: one record split into a 3-bit part and a 4th-bit sidecar, read by rank	live, own reference	MD	T307
out-of-order expert execution in a fixed bitwise order	overlapping expert compute with expert loading (Pre-gated MoE; MoE-Infinity); ours: a fixed summation order that keeps it bit-exact	live, exact	MD	T14
cold ranks half a layer late	KTransformers (used as published)	live at 12 GB	MD	T29

method	builds on	status here	carries	evidence
QOM: drafted rows skip the cold experts that would miss	cache-aware expert skipping (Mixture of Cache-Conditional Experts; MoE-Spec); ours: only on drafted rows, only the experts that would miss, no rerouting	certified config	MD	NOV
3-bit kept experts on drafted rows (U3; "drafter-sure rows" in its tested form)	lower precision for experts on demand (SliceMoE); ours: choosing it by drafted rows	certified config (code sets)	MD	section 8.2
union rule keyed on drafter certainty (Q1-UNION)	OEA; PSR; ours: the union keyed on drafter certainty	RELAX B	MD	T103, T993
relaxed acceptance only inside code comments	relaxed acceptance (Appendix I); ours: restricting it to code comments	certified config (code answers)	SD	section 8.3
copy gate: rows inside a verbatim copy of the prompt count as sure	SAM-Decoding; ours: rows inside the copy count as sure for the lossy row rules	RELAX B	SD, MD	EV:PREREG_ relax_2026-11-02
one extra draft position for the bonus row	the bonus token of speculative decoding (Leviathan et al.); ours: gating it with the next segment's first draft	certified config	MD	F:draft17
per-shape K split for narrow matrix-vector calls (DN-KS)	split-K (standard GEMM practice); ours: the per-shape choice on the M5	certified config	AGPU	[KERNELS]
the request's own prompt footprint as the "cold" table	cache-aware expert skipping (as for QOM); ours: the request's own prompt routing as the table	passed on code, not live	MD	NOV; T312
matrix-unit kernels for padded expert tiles	padded expert GEMMs for few rows (MonoMoE; ONNX Runtime's MoE kernels); ours: the form on the M5's matrix units, bit-exact	live	AGPU	NOV

One idea, reusing the previous position's MoE output, was killed on bytes (section 12.3).

8.1 The SSD wake penalty and the keepalive

- **Builds on.** An NVMe drive enters an idle power state after an idle time and pays a published exit latency on the next request (Microsoft's NVMe power-management guidance for StorNVMe); keeping a drive awake with periodic I/O is old practice (for example Torpid, which writes a small file every 2.5 s); waking a device ahead of predicted I/O is published (HotOS '15). What we found on this MacBook Pro is in section 12.1.
- **Why it works [G].** The penalty depends on the idle gap and not on CPU load (below), so it sits in the SSD path, consistent with an idle timer in the drive or its link. A read every 270 us or sooner keeps that timer from expiring; a single read about 200 us before a burst pays the wake early, off the critical path.
- **Finding.** On this machine's internal SSD, the first read after an idle gap is slower. After gaps of 0.25-1 ms it costs +50-65 us per burst; after 20 ms, +100-120 us [K, T93]. A streamed MoE produces exactly such gaps: one burst of expert reads per layer, with GPU work between bursts. If every layer pays, that is about 3 ms per check at 45 GB [P, T93].
- **A per-call decomposition** puts the fixed cost of one loader call at 187 us of the engine's 283 us: first byte about 119 us, wake 44, dispatch 8, and about 96 us outside the I/O path [K, T97].
- **Fix.** A trickle of 4 KB reads keeps the drive awake. A real spacing of at most 270 us removes the whole +44 us; 400 us or more leaves half [K, T44]. A busy CPU core does nothing, so the effect is in the SSD path [K, T93].
- **In the engine:** a 50 us keepalive, about -3 ms per check on clean prompts [K, BM:docs/NIGHT_ 1104.md:37], adopted in the exact default as part of the +12.1 % bundle (T10). No isolated A/B of the keepalive exists on the final engine (the engine's owner, 6 Nov), so its share of the bundle is known only from that figure.
- **A cheaper form** is a single 4 KB read about 200 us before a predicted burst. It removes 97.5 % of the penalty in microbenchmarks (100 us ahead is too late, 400 us too early), and 87-94 % at the engine's real gaps [K, T47, T98]. It is not live.
- **What does not work:** pinging the record's own first block to pre-warm it (+0.6 to +3.2 us, T202); splitting one record into parallel reads (each piece pays its own request latency, T199); the page cache (+49 us per call, T201).

8.2 Three-bit top-ranked experts on rows the drafter is sure of (as shipped: on every drafted row)

- **Background.** Ranks 0-1 at 4 bits are load-bearing: all ranks at 3 bits fails both the divergence gate and the same-scores test (`TRIED_SHORT.md`; EV:RESULT_relax_2026-11-05, T3ALL: HumanEval+ 88.3 vs 95.0).
- **The signal.** On rows that verify a draft the drafter was confident about, the LSB sidecar is not read for experts that only such rows use. The rationale [G]: on those rows the large model's top token rarely depends on the 4th bit of its top experts. The evidence is the flip rates below.
- **Evidence [K, decision map].** With the union rule at certainty 0.9: 1.19 / 5.06 / 3.24 % flips on full 16-row checks, under the lines (T103). On the QOM form at certainty 0.5: 0.65 / 4.42 / 3.49 %, PASS on all three; at certainty 0 (the shipped setting): 0.71 / 4.45 / 4.14 %, so code and IFEval pass and multilingual chat is BETWEEN (T106). SSD megabytes per kept token: -46.3 % against -34.9 % for the union rule alone [P, T103].
- **As shipped.** The certainty threshold is 0, so every drafted row counts as sure and every kept expert of a drafted row runs at 3 bits; the bonus row stays exact unless it has a draft. The drafter's certainty enters only through the cut that decides which rows are drafted at all (section 4.3). The "certainty 0" numbers above are the shipped setting's.
- **Status.** Part of the certified configuration on the code sets (section 5): same scores as NAX6-N on HumanEval+ and MBPP in both orders [K, F:2026-11-08_0455].
- **Prior art (as recorded).** Per-token mixed precision exists keyed on the router's rank, and per-expert bit widths keyed on importance; none found keys precision on the drafter's certainty (NOV, row "3-bit records on drafter-sure rows").

8.3 Relaxed acceptance restricted to code comments, and its failure when widened

- **The observation.** In code answers, comment text is a small share of positions and a large share of the drafter's misses: 13.5 % of positions and 44.7 % of misses on one walk (T1350); on another, comments carry 52 % of the first misses in a check [P on K, L350 section 1]. A wrong comment word does not change what the code does.
- **The rule** (EV:PREREG_relax_2026-11-02). On a row whose draft sits inside a '#' comment, the draft is kept if it ranks first or second on its row and has at least a quarter of the top token's probability. Everywhere else acceptance is exact.
  - Only inside the answer's code fence; never in its first two lines; never in docstrings, strings or code.
  - Never on a comment that is or could become a tool directive (type comments, linter pragmas, an encoding line, a shebang).
  - The row must already be inside the comment before its draft, and a draft that starts with a newline stays exact.
  - The span is found by an incremental scan of the answer's text, token by token, so it needs no lookahead.
- **In the engine it is safe so far, and small.** As part of RELAX B it passed decision 4 and the blunder bar on code (0 blunders in 25,656 new HumanEval+ tokens), but it adds only 1.8 % to tokens kept per check on HumanEval+ [K, EV:RESULT_relax_2026-11-05]. A replay had promised x1.07 to x1.16 (T1350). The live confidence cut explains the gap: checks that would have continued through a comment are already cut short [P, L350 section 8.2]. In the certified configuration the rule (rank 1-2, at least a quarter of the top probability) is on, and it fired 0 times on the 12 HumanEval prompts of the stack's run [K, F:2026-11-07_0025].
- **Widened, it fails.** "CD16" keeps the drafter's whole comment and docstring text without checking it, and checks only code. On 12 MBPP tasks it kept x1.45 tokens per check and ran +23.7 % faster, and the tasks' tests passed on 8 of 12 against 11 of 12 without it, in both passes: 3 flips against a line of 2 [K, T855; F:45gb-fast-stack]. The answers show comment loops and code that drifts after the drafter's own comments. HumanEval did not show it, because its docstrings copy the prompt. CD16 was dropped the same day, with the headline number that contained it (section 10.3).
- **The lesson for others:** text that "does not matter" still conditions everything after it. Relaxing the choice among the model's own near-top candidates is a different operation from accepting text the model never scored.
- **Tag.** Relaxed and lossy verification is published; the comment-only span was found in neither of two web checks (NOV; INV section 1): adapted.

---

9. Comparison with other engines and prior work

Appendix G has our runs of Strata on this machine, Appendix H the published runs on this model, and the prior art is in Appendix I.

- **Same machine, same prompts [K; Appendix G].** Strata 0.1.58-metal (MIT; github.com/Niko1221/Strata), with its 3-bit IQ3_XXS file, experts streamed from the SSD and its MTP layer, decodes 40.6-47.1 tok/s at a lifetime peak of 40.8-41.2 GB, with its first token in 0.54-1.01 s. Against Strata's arm of the same kind, our lossy decode is x2.5-3.5 its decode on code, x1.7 on math and x1.08 on chat; our exact decode is x1.6-2.2 on code and x1.7 on math, and behind on chat (37.1-38.3 vs 40.6). Our first token is 2.7-7.0x slower, so Strata wins or ties every whole request of up to 256 tokens; ours passes it at about 245-260 tokens on code and math, and at about 340 on chat as shipped [P].
- **Published runs [self-reported; Appendix H].** On our machine class, Splash reports 95 tok/s decode and a 1,312 tok/s prefill with a 2.4-bit file. On a larger Mac (M5 Max, 128 GB) with every weight resident, Sushi reports 138-169 tok/s decode with 2.75-bit weights, and MTPLX 79-126 with 4-bit weights. None of them used our prompts.
- **Prior art [Appendix I].** Every piece of section 7 has published roots: batch invariance (Thinking Machines Lab; vLLM and SGLang, on CUDA), drafters and verification, SSD-streamed MoE inference, expert caching and prefetch, mixed precision for experts, fewer experts under speculation, and relaxed or lossy verification. To our knowledge no published engine for this model combines exact speculative checks, an SSD-streamed rank-tiered store and a certified lossy layer; Strata and Sushi combine several of the pieces.

10. Negative results that teach something

10.0 Headlines we corrected or withdrew

headline as first written	what it was	correction	source
"45 GB FAST v1: 180.3 tok/s" (and 144.9 for the stack before it)	the registered "steady" metric: it timed part of each check and skipped each prompt's first check (~300 ms) and 2.35-4.7 ms of host work per check	142.8 tok/s whole decode, pooled (the note's 142.1 is a geo mean); steady / whole = 1.09-1.27, growing as checks get faster. Since 7 Nov every headline is whole decode	F:2026-11-07_0500; BM:data/r57/s1n{1,2}_256_runs.json
"45 GB FAST stack: 157.9 tok/s"	included CD16, which accepted the drafter's comments unchecked and failed MBPP	withdrawn (section 10.3)	F:2026-11-06_1550 (WITHDRAWN)
"12 GB exact: 40.8 tok/s in 11.35 GB"	the measured arm was a lossy mode; the verified loop runs about 18 tok/s	withdrawn (section 10.5)	F:2026-11-04_1439 (corrected)
"chat runs exact in the lossy config"	the lossy parts that change bits run on every set	chat is lossy, same scores (section 5.2)	F:2026-11-07_2330
our first Strata comparison (whole request, bit tiers)	mixed aggregations; "our exact mode reads 4 bits"	pooled from raw rows; both our modes use the same tiers	F:2026-11-08_1630 (corrected the same day)

10.0b Recent negative results

- **Larger prefill chunks are not exact.** 2,048-8,192-row chunks change the logits (hash equal on 0 of N checks), change decode tokens at 4.7K and 7.9K tokens, and peak at 46.5-53.8 GB [K, F:2026-11-08_0800]. Part of that run was also hit by an MLX bug (sorted gather_qmm past 32,767 rows when M is not a multiple of 64; MLX #4632; [KERNELS]); the 2,048-row arm was not hit and still changes the logits.
- **One MoE call per layer in prefill (PF-B) was not exact at first, and is not exact when a prompt's last chunk is short** (0 of 7-28 logits hashes equal at first, +15-22 %) [K, BM:data/pfb/pfb_report.txt]: the shared expert's one-output gate splits its sums by row count ([KERNELS] section 5.2). Kept per chunk, that gate makes PF-B exact on the three agent prompts of 1.6K-7.9K tokens (8 / 8, 28 / 28, 22 / 22 logits hashes, x1.12-1.21), but it peaks at 45.8-46.2 GB, over the line, and does not ship [K, BM:data/pfd/pfd.log part B]. A short last chunk breaks it again, as in a 30K-token prompt; a fix in a research copy makes those prompts exact (section 6.2).
- **Never waiting for cold experts, third form:** +6.2 % whole decode, pooled (141.4 -> 150.2 tok/s), against a +8 % line: BETWEEN, not adopted [K, BM:data/r57/s1b{b,n}{1,2}_256_runs.json; F:2026-11-07_0550].

- **Gaps chain across references.** Each step passed its own gate (NAX6 -3.3 points against REF2-T, the lossy config -3.3 against NAX6 on 60 HumanEval+ tasks), and the lossy config ends -8.33 points against REF2-T on the HumanEval+ core set [K, F:2026-11-07_0550, F:2026-11-08_0455]. Gates against the immediate predecessor do not bound drift over several steps.
- **Lossy runs are not reproducible token for token** (8 of 12 prompts equal between two passes; section 6.1); a lossy configuration has to be certified under two prompt orders.

10.1 Never waiting for the SSD: passes the quality screen, loses in the engine

- **Idea.** A missing top-2 expert is read in the background and its term is added one layer late, so the GPU never waits (the deferral family, here for the 45 GB FAST mode).
- **Quality screen: PASS.** Code flips 0.68 % against 0.74 % for its twin at a 208-slot pool (T50).
- **Engine: KILL of the first form.** -2.8 % (153.5 vs 157.9 tok/s in the same run, on the steady metric then registered: section 10.0), although kept tokens rose 4.6 %. The check grew from 68.8 to 75.3 ms (T209).
- **Why.** The form did not cut the wait: the gate service went from 14.0 to 15.6 ms per check, and about 6.5 ms of late work was added in the per-layer pipeline (T209c; the engine's owner, 6 Nov). Its own calls still waited on the loader's release for cold optional experts and cold sidecars (LL, 'NEVER-WAIT v3').
- **Why the price was wrong.** The prediction (+10-15 %) treated 13.5-15 ms of gate service per check as removable waiting. It is a spin bound that out-of-order execution already overlaps with compute.
- **Status.** A revised form that drops the cold optional reads is built and bit-checked. It needs its own quality arm before a speed test, so the family is not closed.
- **Note.** Both arms ran on a stack that still contained CD16 (section 10.3). The comparison is within one run.

10.2 Fewer expert groups on sure rows ("split"): the tile price was 2-3x too high

- **Idea.** On rows the drafter is sure of, compute only ranks 0-1 plus experts the check loads anyway.
- **Quality screen.** PASS on code (0.92 % and 1.31 % flips at certainty 0.99 and 0.9, line 1.34 %), KILL on chat (T553, T552).
- **Engine.** Computed expert groups fell to x0.63 and x0.49, yet check + commit fell only 6 and 5 %, and kept tokens fell 4.2 and 1.7 %: net about 0 to +1.5 % against a priced +11-19 % (T610; F:flat2). On the matrix-unit kernel: -1.3 %, with kept tokens -5.7 % (T744).
- **Lesson.** On this GPU the expert time is mostly per-row work, not per-group reads (11.1 ms at 2 rows, 57.0 at 16, all resident). Counting tiles, groups or bytes overpredicted the gain two to three times. A lossy cut also lowers acceptance, and that loss must be in the price.

10.3 Wholesale acceptance of comments (CD16), and a withdrawn 157.9 tok/s

- The stack measured 157.9 tok/s on 12 HumanEval prompts (passes 157.2 / 158.1) and was written up as a headline. It included CD16. CD16 then failed quality on MBPP (section 8.3), and the number was withdrawn before publication (F:45gb-fast-stack, WITHDRAWN).
- **Lesson.** A lever must be gated on a workload that can expose it. HumanEval could not: its docstrings are copies of the prompt, so accepting them unchecked changes nothing. A guarded form (T16) scored BETWEEN on 12 MBPP prompts (one flipped task) and was not adopted [K, F:2026-11-07_0025].

10.4 The 12 GB FAST mode failed same-scores

- **The mode.** A cheap first pass with only the top-2 experts certifies a token when its top probability is at least theta; other rows are recomputed with the full computation.
- **Result.** At theta 0.8 on code and 0.95 on chat: HumanEval+ 80.0 vs 91.7 (0 / 7, p 0.02) and IFEval 84.0 vs 90.0 FAIL; math passes; memory passes (at most 10.84 GB) (EV:RESULT_t0_2026-11-05). Truncations and loops rose from 13 / 2 to 23 / 7.
- **What it teaches.**
  - The decision map had predicted it: the same setting flips 1.63 % of code rows, over the 1.34 % line (T54).
  - The certificate itself is token-accurate (0-1 divergences in about 3,400 tokens at every theta). The damage comes through the context: the cheap pass also writes the attention and recurrent state that later tokens read (T1265). A certificate on the token says nothing about the state it leaves behind.

- **Status.** Since then theta 0.8 and its theta 0.9 fallback failed HumanEval+ in both prompt orders (83.3 and 86.7 vs 95.0 %; EV:RESULT_t80ck1u_2026-11-07). The code-valid point (theta 0.95, map 1.22 %), modelled at 35.9 tok/s [P, a calibrated replay], measured 27.9 tok/s whole decode, pooled, on 12 HumanEval + 12 MBPP [K, F12R:tab12f]. The mode is labelled "lossy, failed its gate".

10.5 A withdrawn "12 GB exact at 40.8 tok/s"

- A public note claimed 40.8 tok/s, bit-exact, in 11.35 GB. A re-check found that the measured arm was a FAST mode: it commits rows certified by the cheap pass without the full check. It matched the verified loop on 5 of 16 prompts, and the quality screen scores its certificate (top-2 experts, theta 0.8) a code KILL at 1.63 %. The verified loop runs 18.3 tok/s whole decode, pooled, on 12 HumanEval + 12 MBPP (whole request 13.1, first token 4.29 s, lifetime peak 11.14 GB) [K, F12R:tab12e]. The note now carries a dated correction (F:12gb-exact-40tps; F12).
- The same week showed that the verified loop itself is not bit-identical to one-token decoding [K, F12 'What the 12 GB exact loop (arm ex) is exact to']. The word "exact" had been used for three different properties: bit-identical to one-token decoding, verified by the full model, and reproducible.
- **Lesson.** An arm's name is not its definition. The exactness check had been run on 4 of the 16 prompts. An audit over every prompt, as in section 3.2, would likely have caught it before the note went out [G].

10.6 Agent turns, and two lossy parts that compound on chat

- **Agent turns.** Plain FAST wrote 85 low-probability tokens in 19,749 on 17 long agent turns (about 43 per 10,000); 75 of them are inside the written file content. Short edits are clean; long file writes carry the failure (EV:RESULT_agentfast_2026-11-06).
- **Cause.** With QOM-KEEP off, blunders fall from 21 to 4 (forward order) and 18 to 0 (reverse): the cache-dependent keep carries them. Other parts do not move the count (EV:RESULT_agentbisect_2026-11-06).
- **Run-to-run dependence is wider than one lever.** With QOM-KEEP off, the two prompt orders still differ within the first tokens. The check logs show the drafted row counts differing about 27 tokens before the committed tokens do, so the drafts carry history. If any FAST kernel's bits depend on a check's row count, even exact acceptance then depends on the drafts (same source, addendum). The exact base must be identical in both orders. It is: every exact run equals one-token decoding, in any order (section 3.2).
- **Chat.** Relaxed acceptance alone scores IFEval 87, 8-bit HC mixers alone 89, both together 84 against 90 for the reference: a failure neither shows alone (EV:RESULT_relax_drift_2026-11-06, section 3).

10.7 A data-aware 2.4-bit trellis store did 1.6-2.5x the damage its error predicted

- Priced from its weight-error ratio against 3-bit truncation, the store measured much worse on the decision map: 1.45 / 9.54 / 10.44 % flips against lines of 1.34 / 5.42 / 3.69 (F:trellis_mse_miss).
- Two independent derivations agree that these errors cost about 1.7-3.5x more KL divergence per unit of expert-output error than truncation errors do. The mechanism is open.
- **Lesson.** For this model, weight or output error cannot rank a data-aware code against plain truncation without a divergence screen on real prompts.

10.8 A fixed cost that did not fall, and a timing method that was wrong

- With every lossy lever of 6 Nov morning on, the fixed cost per check was still 53.7 ms, against a KILL line of 36 ms ([KERNELS] 6.5). 350 tok/s on code at 12 kept tokens would need about 25 ms [P, F:flat2].
- An earlier fit had put a dependent dispatch at 12-20 us, which pointed at fewer, fused kernels. The direct measurement is 1.6 us ([KERNELS] 6.3); the loss was in narrow kernels. Two rebuilt kernels from that programme (a row-parallel GDN step and a replacement HC kernel) were bit-correct and slower than what they replaced (LL, 'GDN-CHUNK', 'HC-K2C').
- A split that was thought live was inert for a night because another fast path bypassed it (F:flat2, caveat). The engaged-path counters of section 3.2 exist for this reason.

10.9 From the ledger: ideas an outside reader would likely propose

One line each; numbers and sources in `TR:TRIED_SHORT.md` and the full ledger.

- **Look-ahead expert prefetch:** pays at 24 GB (+9-10 %), KILL at 45 GB (wrong guesses evict).

- **mmap or the page cache instead of direct reads:** KILL (mmap about 5x slower).
- **Cache-aware routing or substituting resident experts:** KILL; not exact, and small. Expert independence (Appendix F.6) explains why.
- **Fewer experts per token (top-8):** KILL at the strict gate, with or without renormalisation.
- **All experts at 3 bits; any 2-bit tier:** KILL.
- **Draft trees:** KILL; the best tree is -10 to +2 % against the best chain at equal cost.
- **Wider (32-row) checks:** KILL at today's per-row cost (-1.3 %); they pay only below about 0.9 ms per row (KP).
- **Self-speculation, layer skip, early exit:** KILL; exiting after layer 40 agrees with the full model on 5.7 % of code positions.
- **Exact speculative sampling instead of greedy matching:** KILL; acceptance +0.003 to +0.014.
- **Drafter retrains:** on-policy data +0.13 kept per doubling; 2x capacity +0.18; one reseed alone moves code by 0.16. Only the data slope is above noise (KP).
- **Reusing the previous position's MoE output:** the one fully new idea; KILL on bytes (union only x0.91) (NOV).
- **Matrix units for experts at short checks:** KILL at the 1 Nov operating point (+1.8 %); adopted later, once checks were longer and the records were in the kernel's native layout (T675, T527).
- **Compiling the verify graph:** KILL (-0.7 ms; the graph build was not the cost then).
- **Several streams in one pass:** aggregate throughput only; per stream it is slower than solo (F:union-law).

10.10 How far ahead can a forecast help the expert cache? (CPU replays; measurements, no method claim)

A forecast made 8 checks ahead of the experts each layer will need could steer the expert cache. We measured its ceilings on recorded traces: 176 checks of six prompts, replayed on the CPU through the engine's cache at the 45 GB line (K 266 slots per layer) and the 24 GB line (K 147) [K, FAR:far_ceiling.txt; F:2026-11-04_2210_far-ahead-foresight]. No forecaster was built into the engine, and the GPU test that would score a real far draft did not run.

- **The kept path is not enough.** A perfect forecast of only the rows the checks keep cuts misses by 20.3 % against LRU at K 266 (26.3 % at K 147), at recall 0.68 (0.76): the rows the checks reject hold the other 32 % of the reuse [K, FAR:far_ceiling.txt].
- **Misses are not stalls.** Perfect foresight of every row of the next 8 checks cuts misses by 33.7 % but the layers that wait on the SSD by only 5.4 % (Belady's optimum: 55.3 % and 19.3 %) [K, same file].
- **Tokens do not predict deep-layer routes; states do better.** From the token alone, per-row recall@10 of a row's experts is 0.24-0.28 in layers 8-47 (a table built from the other prompts; about 0.40 from the same token's previous occurrence), and the best honest token map cuts misses by 6.0 % [K, same file]. A ridge map from the 125B's own layer-1 MoE inputs of the future rows, standing in for a far drafter's states, reaches recall 0.70 / precision 0.64 and 14.6 % fewer misses at K 266 (stalled layers -1.4 %), and 11.9 % at K 147 [K, FAR:far_ridge.txt]. Both are the best of 16 settings, each fitted leave-one-prompt-out but picked on the same replay; the 16 span 7.9-14.6 % and 6.2-11.9 %.
- **Prior art.** Draft-fed expert forecasts in the literature look ahead one round (SP-MoE, 2510.10302) or at most 32 tokens (Specter, 2511.14102). The state-to-route map itself is adapted from SP-MoE and DraftExpert (2607.24434).

---

11. The path to more speed

Everything we tried is in the public ledger (FableGod/TRIED): one line per attempt, with its verdict, its source and the wall it hit. Below: the gain of each change outside the shipped configuration (11.1), the walls (11.2), and the path for chat and code per memory line (11.3). **Sure** means measured [K] on this engine, in a run that had the change in it; a lossy change is sure as speed, and its quality only once its gate passes. **Bet** means priced [P] from measured parts, or a judgement [G]. Gains multiply only where they were measured in one run.

Today, as shipped at 45 GB (whole decode, 12 prompts per set): HumanEval 164.0, MBPP 112.7, math 78.9 and chat 50.7 tok/s, against 101.4 / 70.4 / 77.9 / 37.1 for the exact default [K, Tables 6.1a-6.1c].

11.1 If this is done, this is the gain

change	line, workload	gain (whole decode unless noted)	mark	state
L by itself (CERT + the exact-safe bundle, without COH)	45 GB, HumanEval / MBPP	+6.0 % / +4.7 % over CERT, pooled, one series (181.3 / 119.8); the shipped 164.0 / 112.7 is another snapshot	[K, BM:data/s5p/s5p_ report.txt]	built; not gated by itself
the chat stack (zs + bundle / HC8_G + rank cut 8 + R1)	45 GB, 12 IFEval	56.0, +12.8 % over the run's first arm; cycle 44.1 ms	[K, BM:data/chs/chs5_ report.txt]	built; not gated
zs32 (M1-B + COH + SB1)	32 GB, 12 IFEval	51.3, against 33.8 for the exact e32 and 39.0 for the shipped lossy-32	[K, BM:data/chs/c32_ report.txt; BM:data/rel/ rel_rows_report.txt]	built; not gated
never waiting for cold experts	45 GB, HumanEval	+6.2 % pooled, under its +8 % line (BETWEEN)	[K, BM:data/r57/ s1b{b,n}{1,2}_256_ runs.json]	built; not shipped
PF-D beyond chat; a better next-layer predictor	45 GB, first token	chat 2.91 -> 2.26 s exact (shipped); lossy form 1.83 s; reading only the missing experts ~1.6 s; 19-31 % of what v4 stages goes unused	[K, Table 6.1c]; 1.6 s [P, BM:data/pfb/pfb_ report.txt]	built for chat; not run on other configurations or at 32 GB
layer-major prefill (exact unless the last chunk is under about 200 tokens)	45 GB, 1.6K-7.9K-token prompts	prefill 1.9-3.4x	[K, F:2026-11-08_0800; LP:data/pfa_tail_ 1111.md]	built; at 45 GB it needs a 0.1 GB memory trim; chat.py's default at 32 and 24 GB (package 1.1.0)
one MoE call per layer in prefill (PF-B)	same	a further 12-21 %, exact on these prompts, not when a prompt's last chunk is short (section 6.2); peaks 45.8-46.2 GB	[K, BM:data/pfd/pfd.log part B; LP:data/prefill_ 1109/exact30k/ summary.txt; LP:data/ tue_afternoon_1110.md]	over the line; the short-chunk fix is in a research copy only
prefill reads overlapped with compute	same	~590 / 750 tok/s prefill at 4.7K / 7.9K	[P, F:2026-11-08_0800]	needs building
two-row verify calls on chat (C2c)	12 GB, 12 IFEval	priced x1.10-1.15 over P2'; measured x1.053 pooled (14.46 vs 13.73, share lock); code x0.881 against A' of an earlier run	[P, F12C]; [K, F12L:chw19_run.log]	screened: chat marginal, code killed
less Python on the critical path	12 GB	at most x1.09	[P, F12P]	test designed, not run
the prompt pass at verify-call bandwidth	12 GB, first token	2.906 s for 63 tokens (21,986 MB at 7.6 GB/s) -> ~1.8 s at ~12 GB/s	[K, F12H]; [P]	needs building

11.2 The limits

Per wall: today's limit; the size of its ledger section (FableGod/TRIED/TRIED.md, one wall per line, read on 9 Nov 2026) and the best result; what breaking it unlocks, and which wall then binds; what could break it.

wall	today's limit	ledger lines; best result	unlocks; then binds	what could break it
fixed cost of a check	chat ~51 ms exact / ~40 ms lossy at 1-2 rows, against a memory floor of ~15-20 ms [K; floor P; F:2026-11-07_ 2355]; the fixed part is ~75-85 % of a chat check [P, [DRAFTER] App. C row 23]	315; warm check + commit 232.6 -> 117.3 ms (T489, T476); chat cycle 48.5 -> 44.1 ms (CHS5) [K]	~2x in the low-row check alone [P]; then kept tokens per check	low-row kernels at bandwidth rather than latency; fewer dependent kernels per layer ([KERNELS]) [G]

wall	today's limit	ledger lines; best result	unlocks; then binds	what could break it
kept tokens per check	chat 2.5, math 6.5, HumanEval 10.7 per check [K, section 4.3]	218; 41 of [DRAFTER]'s 83 items killed; best: the MTP head on chat, +24.5 % [K]	a drafter that never missed: ~105-115 tok/s of chat at today's check cost [P, [DRAFTER]]; then the fixed cost	a chat head trained on ~1.3M tokens of the model's own answers (8 % of that gave x1.014, under its x1.03 line [K, [DRAFTER] 3.1]); Strata's MTP layer accepts 87 % of its drafts [K, F:2026-11-08_1630]
SSD bytes per kept token	45 GB code: 18.5 ms cold-minus-warm per 16-row check [K, F:2026-11-07_0025]; chat misses ~4 ms at 1-2 rows [K, F:2026-11-07_2355]; 12 GB chat 449-491 MB per kept token [K, F12L:chw19_run.log]	303 SSD + 168 bytes; per-layer pools -17.2 % bytes, +8.3 % tok/s, exact; rank tiers about -20 % expert bytes [K, section 7]	at 45 GB code, at most the 18.5 ms per check; then the fixed cost	eviction nearer the offline optimum (OPT: 55.3 % fewer misses and 19.3 % fewer stalled layers than LRU at K 266, 45 GB; section 10.10) [K]; next-layer read-ahead (31-40 ms loader-idle per 12 GB call [P, F12P])
the memory line	rows at 41.7-43.7 GB, at least 1.3 GB under; C 224 -> C 256 +0.1 % [K, F:2026-11-07_0500]	101	little decode at 45 GB; PF-B, 0.8-1.2 GB over the line [K]; at 32 / 24 / 12 GB memory sets the cache, so bytes per kept token bind	trimming PF-B's peak; lossy routes that pass their gates [G]
first token and prefill	chat as shipped 2.26 s, reading 9.6 GB per prompt with PF-D [K, BM:data/pfd/pfd_report_0009.txt]; without it a 37-101-token prompt reads 20.4 GB, though 65.5 % of the routed 3-bit parts are in memory [K, BM:data/pfb/pfb_report.txt]; long prompts compute-bound, SSD at 3.8-5.2 of 14 GB/s [K, F:2026-11-08_0800]	41; PF-D 2.91 -> 2.26 s exact; layer-major 1.9-3.4x [K]	~1.6 s [P]; Strata's 0.54-1.01 s on the same prompts shows more is reachable here [K, section 9]	PF-D everywhere, a better predictor, reads overlapped with compute [P]
quality gates	lossy parts need not compose: relaxed acceptance (IFEval 87) and 8-bit mixers (89) together scored 84, against 90 [K, section 10.6]; CD16 failed MBPP; the 12 GB FAST mode failed same-scores [K, sections 10.3-10.4]	74 quality + 74 exactness	the ungated rows of 11.1; at 12 GB the FAST mode's 44.34 on HumanEval only [K, F12R:c1u16]	lossy rules that pass the gates, gated together [G]

11.3 The path

Today, then 11.1, then each wall. Measured steps are sure; priced steps are bets.

- **45 GB, code (HumanEval / MBPP).** Today 164.0 / 112.7 [K]. Measured, not gated: L by itself, 181.3 / 119.8 [K]. A bet: never waiting for cold experts, +6.2 % on an older configuration (BETWEEN) [K], never run with L. Past that, the SSD wall's room is 18.5 ms per check [K]; nothing further is priced.
- **45 GB, chat.** Today 50.7 [K]. Measured, not gated: the chat stack, 56.0 [K]. Bets: the check at its floor, about 2x in the check alone [P]; a drafter that never missed, ~105-115 tok/s at today's check cost [P]. 150 tok/s needs both walls at once: a ~25 ms cycle and ~3.75 kept per check [P, F:2026-11-07_2355]. First token: 2.26 s today [K], 1.83 s with the lossy form [K], ~1.6 s priced [P].
- **32 GB, chat.** Today 33.8 exact and 39.0 lossy as shipped [K]. Measured, not gated: zs32, 51.3 [K].
- **24 and 12 GB.** The 24 GB line's rows, among them a gated lossy configuration, and its lossy chat route are in [SMALL-RAM]. At 12 GB, code today is 18.34 tok/s verified [K, F12R:tab12e]; the FAST mode's 44.34 (HumanEval only) failed its gate [K, F12R:c1u16], so it is a bet until a lossy rule passes. Chat today is 12.76 verified, 13.97 with P2' (not gated) [K, F12R:chat12, chw14c2]; two-row calls measured x1.053 over P2' (priced x1.10-1.15). Bets, not multiplied: less Python, at most x1.09 [P]; the prompt pass, 2.9 -> ~1.8 s [P]; eviction, bounded by OPT [P].

What is sure and what is a bet. Sure: the shipped rows, and the ungated rows of 11.1 as speed. Every step past a wall is a bet: no wall in 11.2 has been broken in a run, and on chat two must fall together for 150

tok/s. Effort so far: 315 ledger lines on the fixed cost, 471 on SSD bytes, 41 on prefill.

Not measured at all. Concurrent requests (Splash reports 171 tok/s aggregate with 4 concurrent requests on this machine class, self-reported), answers longer than 256 tokens, and prompts longer than 7.9K tokens.

12. What is new in this work

To our knowledge, none of these findings has been published before. For each we give what we found, how we found it, the evidence, why it happens (what is measured [K] and what is still a hypothesis [G]), what it gives, how it could be improved, and where else it could apply. One of the four is an idea that did not pay off (12.3).

12.1 The SSD's wake penalty at sub-millisecond idle gaps

What we found. On this MacBook Pro's internal SSD, the first read after an idle gap of 0.25 ms or more is slower. A streamed MoE runs into this on every layer, because its expert reads come in one burst per layer with GPU work in between. Reads spaced closely enough keep the drive awake and remove the penalty.

How we found it. We timed single reads after controlled idle gaps in micro-benchmarks that load no model, and split the engine's loader cost per call into its parts (section 8.1).

Evidence. After 0.10 ms of idle there is no penalty, and from 0.25 ms on the full one [K, MS:data/ssdmicro.txt]. It costs +50-65 us per burst after 0.25-1 ms gaps and +100-120 us after 20 ms, and a CPU core kept busy changes nothing [K, T93]. A trickle of 4 KB reads at a real spacing of at most 270 us removes the whole +44 us; 400 us or more leaves half [K, T44]. In the engine, a 50 us keepalive saves about 3 ms per check [K, BM:docs/NIGHT_1104.md:37]; it is part of the exact default's +12.1 % bundle, and no A/B isolates it on the final engine.

Why it happens. Measured: the penalty depends on the idle gap and not on CPU load, and reads that end 0.25 ms or more before the next demand read let the drive doze again [K, MS:data/ssdmicro3.txt]. Hypothesis [G]: a low-power state, entered after about 0.10-0.25 ms of idle, that costs about 45-65 us to leave. Still unknown: which part idles (the NVMe controller, the PCIe link, the SoC's storage controller or the kernel's I/O path), and whether a deeper state follows after 20 ms. The mechanism is documented for NVMe drives in general (idle power states with published exit latencies; Microsoft's NVMe power-management guidance for StorNVMe), and keeping drives awake with periodic I/O is old practice (for example Torpid).

What it gives. About 3 ms per check in the engine (above), within the exact default's +12.1 % bundle; per loader call the wake was 44 us of a 283 us fixed cost [K, T97]. It needs only a trickle of small reads, whose energy cost is not measured.

How it could be improved. A single 4 KB read about 200 us before a predicted burst removes 97.5 % of the penalty in micro-benchmarks and 87-94 % at the engine's real gaps, with far fewer reads; it is built but not live [K, T47, T98]. An A/B of the keepalive alone on the final engine is still missing.

Where else it could apply. Untested: any program that reads small bursts with short idle gaps from Apple's internal SSD, such as database index lookups, game asset streaming or data loaders for training. Tested only on this MacBook Pro.

12.2 Small inputs of MLX custom kernels become constants

What we found. MLX's custom-kernel function builds a kernel's signature from the arrays it is called with: a small input is declared `const constant`, a larger one `device`. A kernel written for device pointers then fails to build when it is called with a small placeholder, and a kernel that polls a small input never sees later writes to it.

How we found it. A one-element placeholder input broke the build of a prefill kernel in a run; a probe then varied the input's size ([KERNELS] 5.10).

Evidence. A one-element placeholder was declared `const constant uint32_t*` and the build failed; a 1,024-element placeholder was declared `device`, and the run then matched the path without the kernel (tokens 12 / 12, logits hashes 721 / 721) [K, [KERNELS] 5.10, probe R32; BM:data/pfd/pfd_report_0009.txt line 16]. The polling case, a spin on an array under 4 KB that never released, is in the engine's notes; that run was not saved [BM:FINDINGS.md section 4].

Why it happens. Hypothesis [G]: Metal lets data of 4 KB or less be passed by value and copied when the command is encoded, and small arrays take that path. Still unknown: where MLX sets the size cut ([KERNELS] 5.10).

What it gives. No speed by itself: it turned a build failure into a working kernel. With a 1,024-element placeholder, the PF-D kernel builds and its tokens and logits equal the path without it (12 / 12, 721 / 721) [K, [KERNELS] 5.10], and PF-D brings the shipped chat configuration's first token from 2.91 to 2.26 s (section 4.5). It also argues against polling a small input as a CPU-GPU handoff.

How it could be improved. The engine now tests its kernels with its own placeholders [K, BM:scripts/pfd_unit.py line 147]; where MLX sets the size cut is still to be found in its source ([KERNELS] 5.10).

Where else it could apply. Anyone who writes MLX custom kernels that poll or share state with the CPU, in any field. Tested only in our engine.

12.3 Reusing the previous position's expert output: killed on bytes

What we found. On rows the drafter is sure of, the middle layers' routed-expert output barely changes from one position to the next. Reusing the previous position's output, as a video codec reuses a frame, flips few greedy tokens in layers 12-35. It did not pay off: the experts it saves are not the ones that cost SSD reads. Router-side reuse (for example OEA) changes which experts run, and reuse across the steps of one position is known for diffusion and video models; this reuses an expert's output across adjacent positions.

How we found it. In the decision map, a replay that measures how often a change flips the model's greedy token, reuse was tried per block of 12 layers, then chained as an engine would run it.

Evidence. Flips on drafter-sure code rows: 0.14 % (layers 12-23) and 0.10 % (24-35), against 22.7 % (0-11) and 18.9 % (36-47); layers 12-35 together 1.03 %, between the pass and kill lines; the chained form an engine would run, 8.11 % on code, a kill. The union of experts a check reads shrank only x0.91, so the idea was killed on bytes. Reusing attention or GDN outputs instead flips 29.0 % / 88.5 % [K, the ledger FableGod/TRIED/TRIED.md; the decision map's summaries NIGHT1_SUMMARY.md and NIGHT2_STAGE1.md, not in the public snapshot].

Why it happens. Measured: the middle layers' expert outputs change little between adjacent sure positions, while the first and last 12 layers' do not. Hypothesis [G]: middle layers carry slowly changing context, while early layers track the current token and late layers its prediction. The chained form fails because each reused output feeds the next position's state (as with the FAST mode, section 10.4) [G].

What it gives. Nothing in this engine. The union of experts a check reads shrank only x0.91, about 9 % fewer [P], while the chained form flipped 8.11 % of code rows [K]; SSD bytes bind here (section 11.2), so it was killed. As a negative result it saves others the experiment.

How it could be improved. The per-block form flips only 0.14 % (layers 12-23) or 0.10 % (24-35), one block at a time (1.03 % for both together) [K]; where compute rather than SSD bytes binds, for example with every expert resident, it is the form to test [G]. Not tested.

Where else it could apply. Untested: temporal reuse of conditional-compute outputs in any sequence model with routed layers, such as speech, video or recommendation.

12.4 The M5 matrix unit's fp32 sums match no fixed order

What we found. The M5 GPU's matrix unit accumulates fp32 products in an order that no sequential, pairwise or blocked order reproduces, so an ordinary (ALU) kernel cannot stand in for a matrix-unit kernel bit for bit.

How we found it. An adversarial probe, after the method Fasi et al. (2021) used for NVIDIA's tensor cores, compared the matrix unit's outputs with every fixed summation order we wrote ([KERNELS] 5.6).

Evidence. 1,024 adversarial tests x 512 outputs: the best fixed order matches 45-45.6 %; against the exact sum, 45 % are bitwise, 39 % are 1 ulp off, 14 % 2-8 ulp and 2.4 % more; integer operands match 100 %; the relaxed-precision flag changes nothing [K, [KERNELS] 5.6, probe R12; LP:scripts/nax_order_probe.py].

Why it happens. Measured: integer sums match, so the difference lies in floating-point rounding during accumulation. Hypothesis [G]: the unit adds partial products in a tree with intermediate rounding, or with wider internal accumulators, that no software order reproduces.

What it gives. It is why the exact reference runs on the matrix unit itself (NAX6-N, section 3.1) [G]; that move also brought +6.2 % whole decode (44.74 vs 42.12 tok/s, 13 prompts) [K, section 4.4; LP:data/natfri/natfri_pooled.txt]. It rules out an ALU fallback that is bitwise to the matrix unit.

How it could be improved. A model of the unit's accumulation, as Fasi et al. (2021) built for NVIDIA's tensor cores, would tell an ALU kernel how to match it; not attempted [G].

Where else it could apply. Anyone who needs bitwise reproducibility on Apple's matrix units: scientific computing, deterministic training and inference, verification of numerical code. Tested here only at this engine's shapes.

Appendix C. How the results were produced

- Several sessions worked in parallel on one machine, each owning a part (the engine, kernels, the drafter, the evaluation suite, SSD measurements, the 12 GB line, idea screening). GPU time was serialised by a lock; timings that are compared were taken on a quiet machine.
- Every test has PASS and KILL lines written before it runs; verdicts come from results only. The evaluation suite's pre-registrations and results are dated files (`EvalSuite/docs/PREREG_*`, `RESULT_*`).
- **Software.** MLX pinned to 0.32.2 with the pad-to-64 fix for MLX issue #4632 (sorted gather_qmm past 32,767 rows when M is not a multiple of 64); 0.32.4 changes tokens against our exact references. The bug was reported upstream on 5 Oct 2026, before we hit it on 18 Oct ([KERNELS]). Our engine is released under GPL-3 ([RELEASE]). It builds on the model code of MTPLX by Youssof Altoukhi (Apache-2.0; github.com/youssofal/mtplx), installed from a pinned commit; the release keeps MTPLX's license and NOTICE and shows "Powered by MTPLX" in its startup banner and docs ([RELEASE]). Strata (MIT; github.com/Niko1221/Strata) was run as released in Appendix G; anything taken from it keeps its notice.
- **Memory.** Lifetime peak = the process's maximum phys_footprint (ri_lifetime_max_phys_footprint) in GB of `10^9` bytes; the 45 GB line is 41.9 GiB. It is the engine's own footprint on a 64 GB machine, not an OS limit. Besides it, the runs hold a 7.45 GB volatile expert tier as purgeable memory (section 2).
- **Source keys.** Every source in the text is a key and a path below one of these folders (the data are released with the paper). Paths are relative to the root of the public snapshot, github.com/kamenlevi/350-tps; commit hashes are those of each folder's own repository, and the snapshot holds the files as last committed:

key	file or folder
BM:*file*	the engine's folder (`BigMoE/`)
EV:*name*	an evaluation pre-registration or result, `EvalSuite/docs/<name>.md`; ES:*file* = `EvalSuite/<file>`
F:*slug*	a dated findings note, `FableGod/findings/<slug>.md`
LT:'*heading*'	a section of `Leap/TABLE_350.md`; LL = `FableGod/TRIED/leap.md`
LP:*file*	the kernels' folder, `Leap/<file>`
MS:*file*	`MoonShots/<file>` (tables re-derived for this paper, the related-work check)
SE:*file*	the Strata head-to-head harness and records, `StrataEval/<file>`
T*n*	an entry of the ledger `FableGod/TRIED/TRIED.md` (Appendix D); TR:*file* = `FableGod/TRIED/<file>`
INV / NOV / PA / Q	`NewIdeas/PUBLISH_INVENTORY.md` / `FableGod/TRIED/NOVELTY.md` / `NewIdeas/PRIOR_ART.md` / `NewIdeas/QUEUE.md`
F12	`Fit12/PLAN_12GB_90_fit12.md`, by section heading ('Lever 3 RESULT' when none is given)
F12C	`Fit12/scripts/rows_price.py` (the width-policy replay; inputs chw13 / chw14)
F12P	`Fit12/data/scr/chw16/chw16_prof.txt` and `Fit12/data/scr/chw15/chw15_prof.txt` (reader `Fit12/scripts/chw15_read.py`)
F12H	`Fit12/data/scr/chw18` (chw18_eq.txt, chw18_hist.txt, chw18s.txt)

key	file or folder
F12L:*file*	`Fit12/logs/<file>` (the 12 GB track's run logs)
F12R:*tag*	row *tag* of `Fit12/data/paper/small_ram_12gb_rows.txt` (the 12 GB rows, written from the run logs by `Fit12/scripts/paper_rows.py`)
RF	`MoonShots/research/results_first/RESULTS_FIRST.md` (4 Nov; its 12 GB row is the withdrawn 40.8)
KP / KW / L350	`TrainLab/K_PLAN_1106.md` / `FableGod/goal/K_WALL_1106.md` / `FableGod/full/LOSSY350_FULL.md`
FAR:*file*	`FableGod/far/<file>` (far_ceiling.txt, far_ridge.txt: CPU replays of the expert cache, 4 Nov 2026)

- Every idea, with its verdict and deciding number, is one line in a ledger: 1,345 merged ideas on 6 Nov, of which 142 adopted, 267 passed, 608 killed [P: sums of the section tallies in `FableGod/TRIED/TRIED.md`]. The ledger and the novelty record are the release's appendix (INV section 3).

Appendix D. Ledger keys

A key Tn is a stable label for one entry of the ledger `FableGod/TRIED/TRIED.md`; n was the entry's line number in the merge of Fri 6 Nov ~21:00. To find an entry, search the ledger for its ID. The last column is the entry's line when this table was generated on 6 Nov 2026; `check_sources.py -keys` regenerates it.

key	ledger entry ID	line
T7	Concurrent miss loads	7
T10	Item 5	10
T12	M-17 LAYSZ45	12
T14	OOO + TIER_COMBINE	14
T18	R-69b OOO-LIVE	18
T29	B12LL	29
T44	M-28 SSDMICRO keepalive spacing (4 KB pread + usleep(n), the engine's loop	44
T47	M-29 SSDMICRO2 wake-ahead ping	47
T50	NW-BUILD (the build of NEVERWAIT_TEST.md v2; adapted, RE-OP	50
T54	PLAN-12GB-90 (a plan, not an idea)	54
T76	W1-PICKUP	76
T93	M-20 SSDWAKE	93
T97	M-28 SSDMICRO per-call cost decomposition + JOINT in micro	97
T98	M-29 follow-up	98
T103	Q1U-T3	103
T106	QOM-T3	106
T199	M-28 SSDMICRO pieces	199
T201	M-28 SSDMICRO page cache instead of F_NOCACHE for demand records	201
T202	M-31 HEADPING	202
T209	NW-ENGINE	209
T209c	NW-ENGINE	210
T288	FAR-WAIT	289
T307	REF2-T (R-58)	308
T309	t 0.3 truncation	310
T312	5b U3C len	313
T334	P11	335
T462	U-32	463
T476	GATE16 sync-free 16-row gate	477
T478	L1-23 batch-invariant 16-row dense kernels	479
T479	L1-25 experts_nax	480
T489	R-55 GATE16	490
T490	R-56d	491
T495	Spin gate	496
T527	N7-13 NAX6-N	528
T552	SPLIT-ARMS-CHAT	553
T553	SPLIT-ARMS-CODE	554
T565	Item 2	566
T610	SPLIT-ENGINE-OFF	611
T675	K-62j NAX6	676
T744	SPLIT-NAX	745

key	ledger entry ID	line
T782	D-23d CHATSWITCH	783
T855	CD16-DROP	856
T993	LIVE-CUT AUDIT	994
T1011	CF03-EQ-CF0	1012
T1049	WALL-TIEFLOOR	1050
T1057	Fold-fast with the memory fix (b12llff3, decision 5)	1058
T1088	PURGE-1	1089
T1195	Batch-invariant reframing	1196
T1208	R-40 STEP 4b / 4c -> REF2b key buckets	1209
T1209	R-40 INV1	1210
T1210	R-40 REF2	1211
T1214	R-58 REF2-T	1215
T1218	REF2 sdpa_classes	1219
T1219	REF2b	1220
T1225	MMA-tile exactness check (the kernel owner's claim)	1226
T1244	Chunked (WY / UT) GDN verify	1245
T1259	R-56 REF2 live loop	1260
T1265	12 GB lever 1	1266
T1350	COMMENT-LOOPHOLE	1351

Appendix E. Every row, pooled and geo, per pass

`MoonShots/papers/main_table.txt`, generated by `MoonShots/papers/main_table.py` from `BigMoE/data/r57/<tag>_<C>_runs.json` (read-only), as of 9 Nov 2026. An independent re-derivation of every cell agrees.

```
MAIN paper result table, re-derived from the engine's raw run records (read-only): per config and prompt set,
45 EXACT E (NAX6-N + levers + KEEP + wired), E5 | humaneval | prompts 12 x 2 pass(es) [e5h1,e5h2] | whole
decode pooled 97.8 geo 99.3 (passes 100.0 / 98.6) | whole request pooled 31.7 | first token median 4.59 s
(3.72-6.26) | tokens/task 220
45 EXACT E (NAX6-N + levers + KEEP + wired), E5 | mbpp | prompts 12 x 2 pass(es) [e5m1,e5m2] | whole decode
pooled 68.4 geo 71.2 (passes 69.9 / 72.4) | whole request pooled 21.8 | first token median 3.73 s (3.26-5.60) |
tokens/task 123
45 EXACT EB (E + bitwise Sunday bundle), E5 | humaneval | prompts 12 x 2 pass(es) [e5hb1,e5hb2] | whole decode
pooled 101.4 geo 102.7 (passes 102.3 / 103.2) | whole request pooled 32.6 | first token median 4.41 s
(3.85-6.23) | tokens/task 220
45 EXACT EB (E + bitwise Sunday bundle), E5 | mbpp | prompts 12 x 2 pass(es) [e5mb1,e5mb2] | whole decode
pooled 70.4 geo 73.2 (passes 71.7 / 74.7) | whole request pooled 21.7 | first token median 3.80 s (3.32-4.85) |
tokens/task 123
45 LOSSY S5 v1'+wired (= CERT config), S5 | humaneval | prompts 12 x 2 pass(es) [s5v1,s5v2] | whole decode
pooled 175.3 geo 175.8 (passes 174.6 / 176.9) | whole request / first token: no pfs in 24 of 24 records |
tokens/task 216
45 LOSSY S5 v1'+wired (= CERT config), S5 | mbpp | prompts 12 x 2 pass(es) [s5mv1,s5mv2] | whole decode pooled
111.1 geo 118.3 (passes 118.1 / 118.4) | whole request / first token: no pfs in 24 of 24 records | tokens/task
144
45 LOSSY S5 qall (FAST baseline), S5 | humaneval | prompts 12 x 2 pass(es) [s5q1,s5q2] | whole decode pooled
91.9 geo 92.5 (passes 92.0 / 92.9) | whole request / first token: no pfs in 24 of 24 records | tokens/task 220
45 LOSSY S5 qall (FAST baseline), S5 | mbpp | prompts 12 x 2 pass(es) [s5mq1,s5mq2] | whole decode pooled 65.0
geo 65.8 (passes 66.1 / 65.5) | whole request / first token: no pfs in 24 of 24 records | tokens/task 133
45 LOSSY S5' B (CERT config), S5' | humaneval | prompts 12 x 2 pass(es) [spb1,spb2] | whole decode pooled 171.0
geo 174.0 (passes 168.6 / 179.6) | whole request pooled 36.0 | first token median 4.56 s (4.01-6.40) |
tokens/task 217
45 LOSSY S5' B (CERT config), S5' | mbpp | prompts 12 x 2 pass(es) [spmb1,spmb2] | whole decode pooled 114.5
geo 120.1 (passes 118.3 / 121.9) | whole request pooled 25.8 | first token median 3.79 s (3.43-4.95) |
tokens/task 130
45 LOSSY S5' L = CERT + the bundle (formerly X), S5' | humaneval | prompts 12 x 2 pass(es) [spx1,spx2] | whole
decode pooled 181.3 geo 181.3 (passes 181.7 / 180.9) | whole request pooled 36.6 | first token median 4.51 s
(3.96-6.25) | tokens/task 216
45 LOSSY S5' L = CERT + the bundle (formerly X), S5' | mbpp | prompts 12 x 2 pass(es) [spmx1,spmx2] | whole
decode pooled 119.8 geo 123.6 (passes 122.5 / 124.7) | whole request pooled 25.3 | first token median 3.76 s
(3.37-5.40) | tokens/task 126
45 EXACT EB chat/math, N45 | ifeval | prompts 12 x 2 pass(es) [nie1,nie2] | whole decode pooled 37.1 geo 39.9
(passes 39.9 / 39.8) | whole request pooled 20.8 | first token median 2.89 s (2.23-3.88) | tokens/task 141
45 EXACT EB chat/math, N45 | math500 | prompts 12 x 2 pass(es) [nme1,nme2] | whole decode pooled 77.9 geo 78.8
(passes 79.4 / 78.3) | whole request pooled 38.9 | first token median 2.73 s (2.52-6.75) | tokens/task 256
45 LOSSY L = CERT + the bundle, chat/math, N45 | ifeval | prompts 12 x 2 pass(es) [nil1,nil2] | whole decode
pooled 43.4 geo 45.0 (passes 45.2 / 44.8) | whole request pooled 21.1 | first token median 3.00 s (2.43-3.93) |
tokens/task 128
45 LOSSY L = CERT + the bundle, chat/math, N45 | math500 | prompts 12 x 2 pass(es) [nml1,nml2] | whole decode
pooled 86.9 geo 87.8 (passes 85.3 / 90.4) | whole request pooled 39.6 | first token median 2.96 s (2.71-6.98) |
tokens/task 256
```

```
45 EXACT E + COH (chat), CHS4 ycoh / ycoh2 | ifeval | prompts 12 x 2 pass(es) [ycoh,ycoh2] | whole decode
pooled 37.9 geo 40.1 (passes 39.9 / 40.3) | whole request pooled 21.2 | first token median 2.83 s (2.17-3.79) |
tokens/task 141
45 EXACT E + COH + SB1 (chat), CHS4 yes1 | ifeval | prompts 12 x 1 pass(es) [yes1] | whole decode pooled 38.3
geo 40.6 (passes 40.6) | whole request pooled 21.3 | first token median 2.82 s (2.34-3.71) | tokens/task 141
45 LOSSY M1-B + COH (chat), CHS5 z0 / z0b | ifeval | prompts 12 x 2 pass(es) [z0,z0b] | whole decode pooled
49.8 geo 53.7 (passes 53.4 / 54.1) | whole request pooled 24.4 | first token median 2.92 s (2.34-3.81) |
tokens/task 145
45 LOSSY M1-B + COH + SB1, CHS5 zs | ifeval | prompts 12 x 1 pass(es) [zs] | whole decode pooled 51.9 geo 55.8
(passes 55.8) | whole request pooled 24.8 | first token median 2.98 s (2.51-3.65) | tokens/task 145
45 LOSSY stack M1-B+COH+SB1+bundle/HC8_G+rank cut 8+R1 (ungated), CHS5 zsgcr | ifeval | prompts 12 x 1 pass(es)
[zsgcr] | whole decode pooled 56.0 geo 59.9 (passes 59.9) | whole request pooled 24.6 | first token median 2.94
s (2.55-3.74) | tokens/task 134
45 LOSSY chat as shipped: zs + PF-D v4 + read-ahead, pdz4a | ifeval | prompts 12 x 1 pass(es) [pdz4a] | whole
decode pooled 50.7 geo 54.3 (passes 54.3) | whole request pooled 28.8 | first token median 2.26 s (1.55-2.59) |
tokens/task 145
45 LOSSY zs, control without PF-D, pdz0 | ifeval | prompts 12 x 1 pass(es) [pdz0] | whole decode pooled 50.3
geo 53.7 (passes 53.7) | whole request pooled 24.6 | first token median 2.91 s (2.34-3.74) | tokens/task 145
45 LOSSY L + COH as shipped (snapshot rel1), REL | humaneval | prompts 12 x 1 pass(es) [rlh] | whole decode
pooled 164.0 geo 164.3 (passes 164.3) | whole request pooled 36.2 | first token median 4.60 s (3.78-6.07) |
tokens/task 218
45 LOSSY L + COH as shipped (snapshot rel1), REL | ifeval | prompts 12 x 2 pass(es) [rli,rlr] | whole decode
pooled 45.2 geo 45.5 (passes 45.8 / 45.3) | whole request pooled 22.4 | first token median 3.01 s (2.41-3.81) |
tokens/task 138
45 LOSSY L + COH as shipped (snapshot rel1), REL | math500 | prompts 12 x 1 pass(es) [rlt] | whole decode
pooled 78.9 geo 79.9 (passes 79.9) | whole request pooled 37.9 | first token median 3.20 s (2.71-6.80) |
tokens/task 256
45 LOSSY L + COH as shipped (snapshot rel1), REL | mbpp | prompts 12 x 1 pass(es) [rlm] | whole decode pooled
112.7 geo 116.0 (passes 116.0) | whole request pooled 25.3 | first token median 3.72 s (3.35-4.88) |
tokens/task 127
32 EXACT e32 (C 160), C32 | ifeval | prompts 12 x 1 pass(es) [e32] | whole decode pooled 33.8 geo 34.5 (passes
34.5) | whole request pooled 19.8 | first token median 2.83 s (2.24-3.62) | tokens/task 141
32 LOSSY zs32 (C 160), C32 | ifeval | prompts 12 x 1 pass(es) [zs32] | whole decode pooled 51.3 geo 54.0
(passes 54.0) | whole request pooled 22.5 | first token median 2.99 s (2.47-3.84) | tokens/task 124
32 LOSSY L + COH as shipped at C 160 (snapshot rel1), REL | ifeval | prompts 12 x 1 pass(es) [rli_160] | whole
decode pooled 39.0 geo 37.5 (passes 37.5) | whole request pooled 21.1 | first token median 2.90 s (2.37-3.81) |
tokens/task 138
```

Appendix F. Measurements on the Apple M5 GPU (summary; the full form is [KERNELS])

All timings are from one M5 Pro; "quiet" means no other GPU job.

- **F.1 One dependent dispatch costs about 1.6 us** (1.24-1.56 us per kernel in a chain of 200), and a check launches about 1,000-4,000 kernels, so dispatch costs about 1-6 ms per check, not the 20 ms an earlier fit assumed [K per launch, LT:'STEP 0 PART B RESULT'; P for the check; [KERNELS] 6.3].
- **F.2 The time is lost in narrow matrix-vector kernels;** a per-shape K split (DN-KS) recovers much of it, at rounding level ([KERNELS] 6.4).
- **F.3 Single-call attention** (bitwise below 512 keys) and a 17-32-row drafter block brought the 2-row fixed cost from 55.2 to about 42-44 ms in one day [K, LL, 'SDPA-1'; [KERNELS] 4.4].
- **F.4 Where a check's time goes.** With every lossy lever of 6 Nov morning, the fixed cost of a check (the steady 2-row cycle minus twice the warm per-row slope) was 53.7 ms: a KILL against the pre-registered lines (PASS at 30 ms, KILL above 36 ms) [K, F:flat2]. The split by piece is in [KERNELS] 6.5.
- **What it caps [P].** At 12 tokens kept per check, a 54 ms fixed cost alone caps code near 220 tok/s even with free experts. Speed on code is `kept / (fixed + per-row x rows + SSD wait)`; all three terms and the kept count must move together.
- **Kept tokens depend on the workload [K, KP].** 10.36 per check on the engine's 12 HumanEval prompts; on exact walks 9.57 (HumanEval+), 5.94 (MBPP), 7.00 (GSM8K) and 3.01 (agent turns). On the 12 HumanEval prompts at a 256 token cap, 67.8 % of generated tokens are one verbatim copy of the prompt's signature and docstring [K, F:lossy350-thinking-hour; L350 section 1]; MBPP, math and IFEval answers copy 1-3 %. A code tok/s number without its workload is not comparable.

F.5 CPU-GPU visibility on unified memory

- A GPU kernel that spins on shared memory sees CPU writes only after 0.5-1,000 ms with relaxed or device-scope atomics; visibility is guaranteed only at command-buffer boundaries [K, BM:FINDINGS.md

section 4]. Small inputs (under 4 KB) are copied at encode time, so a kernel spins on a snapshot forever (from the engine's notes; the run behind it was not saved).
- Two forms work: the CPU releases a layer by writing a fresh region that the GPU polls line by line (never-read lines cannot be stale; the engine's notes give about 69 us per gated layer against about 230 us for a full synchronisation, from runs that were not saved [BM:FINDINGS.md section 4; T495]), and system-coherent loads, which bring the loader-to-GPU pickup to a median of 2.9 us [K, T76].
- In the other direction, device-scope GPU stores reach the CPU only on flush, about 20-23 us after the next dispatch [K, T76].
- Read-ahead hints on the SSD slowed the GPU by 73 % in another group's report, and look-ahead prefetch did not pay at 45 GB here (section 10.9): on this hardware SSD traffic and the GPU share the memory system.

F.6 The model's own structure

- **Experts are independent functions.** On the real weights (three layers): no shared base, no common subspace, no near-duplicate neurons; routed expert outputs on one input are near-orthogonal (mean cosine +0.02). Substituting a resident expert for a missing one equals dropping it [K, F:lossy350-thinking-hour].
- **A union law.** If every row of an R-row check is kept, distinct (expert, layer) groups per kept token fall only about 25 % per doubling of R: 276 / 218 / 163 on code at R 8 / 16 / 32 [P on K routing, F:union-law]. Wider checks cannot make experts cheap by sharing.
- **Chat is an acceptance problem.** At equal check length chat routes fewer experts per token than code. Its cost comes from short checks (about 2-4 rows) and rejected rows [P on K, same].

Appendix G. Same machine, same prompts: Strata (detail of section 9)

Setup [K, F:2026-11-08_1630_strata-head-to-head and its two reports]. Strata 0.1.58-metal (commit 146df3c; MIT; github.com/Niko1221/Strata), its IQ3_XXS file (3-bit, 75.8 GB on disk; huggingface.co/ISTA-DASLab/Qwen3.8-Flash-Next-GSQ-RCO-GGUF; we tested only this file, and its 4-bit files are untested), experts streamed from the SSD, on this machine, the same 48 prompt ids as Table 6.1, greedy, 256 new tokens, EOS stop, one warm process per arm and pass. Arm D reads its hyper-connection weights exactly (BF16), arm B reads them as Q8_0 (lossy, its default); both use its MTP layer. Full cache: arena 33 GB (the size Strata picks on a free Mac), one pass per arm, lifetime peak 41.15 GB (D) and 40.76 GB (B), against our 41.7-43.2 GB.

Table G.1. Whole decode / whole request / first token (median), pooled [K].

set	ours, exact (EB)	ours, lossy as shipped (L + COH)	Strata D (exact HC read)	Strata B (Q8_0 HC read)
12 HumanEval	101.4 / 32.6 / 4.41 s	164.0 / 36.2 / 4.60 s	46.4 / 40.6 / 0.66 s	47.0 / 40.9 / 0.67 s
12 MBPP	70.4 / 21.7 / 3.80 s	112.7 / 25.3 / 3.72 s	43.6 / 35.8 / 0.54 s	45.6 / 36.9 / 0.54 s
12 IFEval (chat)	37.1 / 20.8 / 2.89 s	45.2 / 22.4 / 3.01 s	40.6 / 33.0 / 0.83 s	41.8 / 33.7 / 0.83 s
12 math	77.9 / 38.9 / 2.73 s	78.9 / 37.9 / 3.20 s	46.3 / 39.9 / 1.01 s	47.1 / 40.5 / 1.01 s

- Strata's columns are re-derived from its 96 request records: whole decode = generated tokens / decode time, whole request = generated tokens / request wall time (sent to done), both pooled; first token = the median of sent to first token (all match its report) [K, SE:h2h/results_arena33/{B1,D1}.requests.jsonl]. Its peaks are /usr/bin/time's peak memory footprint, the same counter and unit (`10^9` bytes) as ours.
- Our lossy column is the configuration we ship (Table 6.1b; run 9 Nov on the same prompt ids). CERT decodes faster on code (171-175 and 111-114 tok/s) but has no chat or math rows of its own; L's are in Table 6.1c.
- **Chat as shipped (zs + PF-D v4; lossy, same scores on IFEval and the card's benchmarks; Table 6.1c, a later run, one pass) [K]:** 50.7 / 28.8 / 2.26 s at 43.7 GB, against Strata's 40.6-41.8 / 33.0-33.7 / 0.83 s: decode x1.2 its Q8_0 arm's, whole request behind (x0.85), first token 2.7x slower.
- **Where we win:** decode, each against Strata's arm of the same kind (our exact against D, our lossy against B): x1.6-2.2 exact and x2.5-3.5 lossy on code, x1.7 exact and x1.7 lossy on math, x1.08 lossy on chat.

- **Where we lose:** the first token, 2.7-7.0x slower (medians per set; 2.7x on chat as shipped, with PF-D); every whole-request number up to 256 tokens (Strata wins or ties math); and exact chat decode (37.1-38.3 vs 40.6). Lossy chat as shipped is slightly ahead (45.2 vs 41.8; L's earlier runs 43.4-44.2), and our gate-open chat lever M1-B reaches 49.8 (Table 6.1c).
- **Break-even [P: first-token medians and pooled decode].** Ours passes Strata's whole-request time at about 260 tokens on HumanEval, 245 on MBPP and 255 on math (lossy as shipped), at about 1,200 on chat with it (L's earlier runs: 1,750-2,500), at about 550 on chat with M1-B and at about 340 on chat as shipped (zs with PF-D), each against Strata's Q8_0 arm.
- **Strata's own levers [K, same note]:** its MTP layer gives +53 % decode at its default cache (25.3 -> 38.7 tok/s pooled, all sets; 87 % of drafted tokens accepted); its Q8_0 read adds 2-3 % pooled over its exact read (1-5 % per set); a larger cache (25-28 -> 33 GB) adds 12-23 % per set.
- **Equal memory, our 32 GB line (details in [SMALL-RAM]) against Strata's default cache (peak 33.5-34.9 GB), chat [K]:** ours exact 33.8 / 19.8 / 2.83 s at 30.6 GB (12 / 12 token-identical to one-token decoding), ours lossy ungated 51.3 / 22.5 / 2.99 s at 31.2 GB; the shipped lossy-32 (L + COH at C 160; its card gate ran at C 256 only, and its IFEval tokens equal C 256's on 12 / 12) 39.0 / 21.1 / 2.90 s at 31.3 GB [K, BM:data/rel/rel_rows_report.txt]; Strata 35.8 (exact read) and 37.3 (Q8_0) decode, 28.2 and 29.5 whole request, first token 1.06 s (exact read) and 0.98 s (Q8_0) (BM:data/chs/c32_report.txt; e32, zs32 in Appendix E; Strata's from SE:h2h/results/{B,D}{1,2}.requests.jsonl). PF-D was not run at 32 GB.
- **Caveats [K, same note].** Different files: Strata's 3-bit IQ3_XXS against our tiered store (ranks 0-1 at 4 bits, 2-9 at 3, in both our modes). Quality was not compared between the engines; 2 of 24 rows are token-identical to our exact reference on HumanEval and on chat, none on MBPP or math. Strata's full-cache arms are one pass each (its default-cache passes repeat within 5 %). Long prompts (1.6K+ tokens) were not compared. Our code and math rows predate the layer-major prefill.

Appendix H. Published runs on this model (all self-reported unless noted)

Every row was re-read from its source on 8-9 Nov 2026 (`MS:research/relwork/RELWORK_CHECK.md`). None of these states whole request and first token for the same runs as its decode figure, and none used our prompts, except our own Strata run (Appendix G).

who (source)	expert weights	exactness as stated	memory as stated	hardware	decode tok/s as stated	other
Splash (README "Performance"; release 1.4.0 notes)	Q2_0, 2.4 bits per weight	not stated for these rows	not stated	**M5 Pro, 20-core GPU, 64 GB**	95 (short coding prompts, 1,024-token outputs)	prefill 1,312 tok/s at 32K; cached first token 171 ms on a 32K replay; 4 concurrent: 171 aggregate
Strata, as measured by Splash (same README)	Q2_0	-	-	same	about 73 [P: 95 / 1.3]	-
Strata, our run (Appendix G)	IQ3_XXS, 3 bits	exact or Q8_0 hyper-connection read	lifetime peak 40.8-41.2 GB	our M5 Pro, 64 GB	40.6-47.1 [K]	first token 0.54-1.01 s [K]
Kapitsa (README)	Q2_0, streamed from the SSD	"the big model checks every token"	48 GB machine	M5 Pro, 48 GB	61	also 113.9 greedy on an M5 Max (Nov)
Sushi 1.4.0 (github.com/beamivalice/sushi, README "Benchmarks"; MIT)	STQ trellis quant, 2.75 bits per weight (2 and 4 bpw packs too)	not stated	weights 44.9 GiB, resident ("requires 64 GB+"); its 2 bpw pack streams at 32 GB	M5 Max, 128 GB	138.2-168.6 (4K-128K context); 4 bpw: 122.0-153.5	prefill 1,122-2,049 tok/s; MTP drafting
Mference (github.com/NeelM0906/Mference, README)	not stated	not stated	resident	M3 Ultra	18.2-21.0	prompt 24-210 tok/s

who (source)	expert weights	exactness as stated	memory as stated	hardware	decode tok/s as stated	other
MTPLX 2.11.3-2.12.0 (github.com/youssofal/MTPLX, README; Apache-2.0; our engine builds on its model code)	4-bit (flat, or dynamic with 8-bit attention)	"exact rejection sampling", "correct at any temperature"; token identity with one-token decoding not stated	resident: peaks 78-87 GiB ("96 GB and up", "128 GB and up"); the n-gram table streams from the SSD	M5 Max, 128 GB	79.3 (9K-token code prompt, 1,500 tokens); 125.8 (one agent request, 18.4K of 18.5K prompt tokens cached); 50.3-63.5 at 65K-200K tokens	prefill 1,094 tok/s at 65K; MTP drafting, depth 3
SlotStream (README)	original MLX 4-bit	exact	33 GB budget	M5 Pro, 48 GB	20.91-23.69	-
Evolving Agent Teams (arXiv 2610.36848)	MLX 4-bit, streamed	not stated	48 GB machine	M4 Pro, 48 GB	30.3	-
Palm-Infra (README; docs/ssd-offload.md; Apache-2.0)	NVFP4 experts, CPU SSD offload	not stated	20 GiB expert cache	CPU, eight threads (its docs: a 48 GB Apple Silicon Mac)	21.8, best observed; a controlled four-thread run with a 16 GiB cache: 15.74 (median of five processes)	prefill 16.2 best observed, 18.70 in the controlled run; a CPU-side engine
this work, 45 GB line (Table 6.1)	about 3.2 bits	exact to NAX6-N; lossy as shipped (L + COH): same scores on the card's IFBench, GPQA Diamond and LiveCodeBench v6; CERT: same scores on HumanEval+, MBPP, math and IFEval; chat as shipped: lossy, same scores on IFEval and the card's benchmarks	lifetime peak 41.7-43.7 GB	M5 Pro, 64 GB	exact 37.1-101.4; lossy as shipped 45.2-164.0; CERT 111.1-175.3 on code [K]	first token 2.7-4.6 s; 2.26 s on chat as shipped (PF-D) [K]

- **What this table can and cannot say.** The prompts, answer lengths, files and memory settings differ from row to row. Splash's row is on our machine class with a 2.4-bit file and coding prompts; on those terms it reports a faster decode than our exact mode on code and than every chat number of ours, and a far faster prefill and first token. Our lossy decode on HumanEval and MBPP (113-164 tok/s as shipped) is above every other decode figure from an M5 Pro in the table, but HumanEval flatters (section 6) and no run of Splash on our prompts exists: its figures are self-reported, on its own prompts and file. Sushi's 138-169 tok/s decode and 1.1-2.0K tok/s prefill come from a larger machine (M5 Max, 128 GB) with every weight resident; its 2.75-bit weights alone (48.2 GB) are above our 45 GB line, and its 2-bit pack is the one it streams. MTPLX's 79.3-125.8 tok/s are from the same larger machine with 4-bit weights resident (peaks 78-87 GiB, about twice our line).
- **Other >= 80B MoE rows, re-opened.** Memory-Sovereign Inference (arXiv 2608.23805) streams Qwen3-Next-80B in Q4_K_M with bit-identical logits; its 11.6 tokens/s is "a mechanism diagnostic", not a throughput claim. PagedMoE (arXiv 2611.03173) reaches 8.89 tokens/s on Qwen3.5-397B with a 24 GiB expert cache on a PC with an RTX 5090.
- **Coverage.** Engines and papers are as found by the check of Sun 8 Nov and Mon 9 Nov (`research/relwork`); later releases are not covered.

Appendix I. Prior art

Items are grouped by what we took from them; the full entries are in the References. Every arXiv identifier was checked against its title and authors through the arXiv API on 8-9 Nov 2026, and every web source was re-read then.

Batch invariance and determinism. H. He and Thinking Machines Lab, "Defeating Nondeterminism in LLM Inference", Connectionism, Sep 2025 (doi 10.64434/tml.20250910; batch-invariant RMSNorm / matmul / attention, fixed split sizes); its batch_invariant_ops library (MIT; github.com/thinking-machines-lab/batch_

invariant_ops); vLLM's batch invariance mode (VLLM_BATCH_INVARIANT, vLLM docs) and SGLang's deterministic inference (LMSYS blog, "Towards Deterministic Inference in SGLang and Reproducible RL Training", 22 Sep 2025), both CUDA only; LLM-42 (arXiv 2601.17768). Our port and the three MLX-specific breaks are section 3 and [KERNELS]. Our early notes did not cite this work; they should have (PA, row 1).

SSD-streamed MoE inference. Memory-Sovereign Inference (arXiv 2608.23805): output-exact NVMe streaming of an 80B model with byte-identical logits. Public engines: Strata (MIT; github.com/Niko1221/Strata), Splash (github.com/incoai/splash), Kapitsa (github.com/nace-ai/kapitsa), SlotStream (github.com/carloslfu/slotstream), anemll-flash-mlx (slot banks, expert ids as data), Flash-MoE, Palm-Infra (github.com/TencentYoutuResearch/Palm-Infra), Sushi (MIT; github.com/beamivalice/sushi), Mference (github.com/NeelM0906/Mference). Evolving Agent Teams (arXiv 2610.36848); PagedMoE (arXiv 2611.03173). Alizadeh et al., "LLM in a flash" (arXiv 2312.11514). KTransformers (Chen et al., SOSP '25, doi 10.1145/3731569.3764843): Expert Deferral.

Expert caching and prefetch. Pre-gated MoE (Hwang et al., 2308.12066); ProMoE (2410.22134); MoE-Infinity (2401.14361); AdapMoE (2408.10284); HOBBIT (2411.01433); FlashMoE (2601.17063); SwapMoE (2308.15030); Specter (2511.14102); SP-MoE (2510.10302); Fiddler (2402.07033).

Mixed precision for experts. HOBBIT; MC-MoE (2410.06270); bandwidth-efficient adaptive MoE via low-rank compensation (2512.17073); DynaExq (2511.15015); DyMoE (2603.19172); SliceMoE (2512.12990); Matryoshka Quantization (2502.06786); QTIP trellis codes (2406.11235).

Fewer or shared experts under speculation. OEA, Opportunistic Expert Activation (2511.02237); PSR, Prefetch-Steered Routing (2610.34721); AcceptMoE (2608.02989); MoE-Spec (2602.16052); EVICT (2605.00342); DraftExpert (2607.24434); EcoSpec (2607.12696); Utility-Driven SD for MoE (2506.20675); Lynx (2411.08982); Mixture of Cache-Conditional Experts (2412.00099).

Drafters and verification. Leviathan et al., "Fast Inference from Transformers via Speculative Decoding" (2211.17192; the cost ratio); Chen et al., speculative sampling (2302.01318); DFlash (Chen, Liang and Liu, "DFlash: Block Diffusion for Flash Speculative Decoding", ICML 2026; 2602.06036; github.com/z-lab/dflash, MIT); DFlash2, the base of our fine-tuned block drafter ([DRAFTER]): Inco AI, "DFlash 2: Keep Drafting Parallel" (blog, Aug 2026, inco.ai/blog/dflash2), checkpoint incoai/Qwen3.8-Flash-Next-DFlash2 (Hugging Face; Apache-2.0; a finetune of Qwen/Qwen3.8-Flash-Next; revision a22a174 of 19 Oct 2026; the engine loads it and our fine-tuned weights over it); MTPLX by Youssof Altoukhi (software, v2.12.2 at commit 9882703f, the version we ran; Apache-2.0; github.com/youssofal/mtplx: the model's own MTP head as the drafter on Apple Silicon, with exact rejection sampling; its first backend for this model family is dated 29 Aug 2026; our engine builds on its model code); DFlare (2606.02091); EAGLE-3 (2503.01840); PARD-2 (2605.08632); WhiFlash (2606.07710; switches between an autoregressive and a diffusion drafter token by token); TriSpec (2601.23180); SAM-Decoding (2411.10666); prompt-lookup decoding (github.com/apoorvumang/prompt-lookup-decoding).

Relaxed and lossy verification. MARS (2601.15498); DIVERSED (2604.07622); "Revisiting Lossy Verification" (2607.26627); AdaptiveSpec (2609.02897); the answer-lengthening effect of relaxed rules (2610.23818).

Kernels. MLX (Apple; MIT): issue #4632 (sorted gather_qmm past 32,767 rows, reported 5 Oct 2026) and PR #3922 ("Fix sorted gather_qmm NAX row overflow above 32K", merged 26 Aug 2026); details in [KERNELS]. MonoMoE (2609.04244, padding waste in decode MoE matmuls); ONNX Runtime's MoE small-row path; QuaRot (2404.00456) / SpinQuant (2405.16406) (folding a norm weight into the next linear). Apple describes the M5 GPU as having "a Neural Accelerator in each core" (Apple Newsroom, Oct 2025); how MLX reaches them is in [KERNELS].

Storage. The exit latency of an NVMe link's low-power state (SK Hynix, US 11,789,658 B2); waking sleeping hardware ahead of use (Zhu et al., "Software Engagement with Sleeping CPUs", HotOS '15); direct I/O into device-visible buffers (NVIDIA GPUDirect Storage; BaM, Qureshi et al., 2203.04910).

Benchmarks. HumanEval (Chen et al., 2107.03374); HumanEval+ and MBPP via EvalPlus (Liu et al., 2305.01210); MBPP (Austin et al., 2108.07732); IFEval (Zhou et al., 2311.07911); MATH (Hendrycks et al., 2103.03874) and its 500-problem subset MATH-500 (Lightman et al., 2305.20050); GSM8K (Cobbe et al., 2110.14168).

The model. Qwen3.8-Flash-Next (Hugging Face: Qwen/Qwen3.8-Flash-Next); Qiu et al., "On the Design of Qwen3.8-Next Architecture: Evaluation, Efficiency, and Training Stability" (2608.30320).

Corrections

- **10 November 2026 (v1.0.1).** The abstract's memory line now applies to the benchmark prompts: longer prompts go over it (section 6.1). PF-D's exactness on 10K- and 30K-token prompts was added (section 4.5). PF-B's exactness is now limited to the prompts it was checked on, up to 7.9K tokens; on a 30K-token prompt it is not exact (section 6.2).
- **11 November 2026 (v1.1.0).** [SMALL-RAM] is published. PF-B's difference on a 30K-token prompt has a found cause, a short last chunk, and a fix in a research copy, not in the package (section 6.2). The layer-major prefill, which runs only on prompts over 1,024 tokens, is exact only when their last 1,024-token chunk is about 200 tokens or longer; a shorter one changed the tokens (sections 3.2, 4.5, 6.2, 7 and 11.1). It now ships in package 1.1.0, as chat.py's default at 32 and 24 GB (sections 4.5, 7 and 11.1).

Use of AI tools

The code, the experiments and the text of this paper were produced with substantial help from AI coding agents. The author directed the work and is responsible for its content. The numbers were re-derived from the raw run records named beside them, and the references were opened by hand or by script.

References

- Keivan Alizadeh et al. "LLM in a flash: Efficient Large Language Model Inference with Limited Memory." arXiv:2312.11514, 2023.
- Youssof Altoukhi. MTPLX (software, v2.12.2, commit 9882703f; Apache-2.0). https://github.com/youssofal/mtplx
- Zihao An et al. "PARD-2: Target-Aligned Parallel Draft Model for Dual-Mode Speculative Decoding." arXiv:2605.08632, 2026.
- Anemll. anemll-flash-mlx (software). https://github.com/Anemll/anemll-flash-mlx
- Apple. "Apple unleashes M5, the next big leap in AI performance for Apple silicon." Apple Newsroom, 15 Oct 2025. https://www.apple.com/newsroom/2025/10/apple-unleashes-m5-the-next-big-leap-in-ai-performance-for-apple-silicon/
- Saleh Ashkboos et al. "QuaRot: Outlier-Free 4-Bit Inference in Rotated LLMs." arXiv:2404.00456, 2024.
- Jacob Austin et al. "Program Synthesis with Large Language Models." arXiv:2108.07732, 2021.
- Shiyi Cao et al. "Evolving Agent Teams." arXiv:2610.36848, 2026.
- Charlie Chen et al. "Accelerating Large Language Model Decoding with Speculative Sampling." arXiv:2302.01318, 2023.
- Hongtao Chen et al. "KTransformers: Unleashing the Full Potential of CPU/GPU Hybrid Inference for MoE Models." In Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles (SOSP '25), pp. 1014-1029, 2025. doi:10.1145/3731569.3764843.
- Jian Chen, Yesheng Liang and Zhijian Liu. "DFlash: Block Diffusion for Flash Speculative Decoding." arXiv:2602.06036, 2026. ICML 2026. Code: https://github.com/z-lab/dflash (MIT).
- Liangkun Chen et al. "SP-MoE: Speculative Decoding and Prefetching for Accelerating MoE-based Model Inference." arXiv:2510.10302, 2025.
- Mark Chen et al. "Evaluating Large Language Models Trained on Code." arXiv:2107.03374, 2021.
- Yuseon Choi et al. "SliceMoE: Bit-Sliced Expert Caching under Miss-Rate Constraints for Efficient MoE Inference." arXiv:2512.12990, 2025.
- Kexin Chu et al. "Dynamic Expert Quantization for Scalable Mixture-of-Experts Inference." arXiv:2511.15015, 2025.
- Karl Cobbe et al. "Training Verifiers to Solve Math Word Problems." arXiv:2110.14168, 2021.
- Flash-MoE (software). https://github.com/danveloper/flash-moe
- Raja Gond et al. "LLM-42: Enabling Determinism in LLM Inference with Verified Speculation." arXiv:2601.17768, 2026.
- Yu Gong et al. "MonoMoE: An Efficient Fused Mega-kernel for Quantized MoE Decoding." arXiv:2609.04244, 2026.
- Vima Gupta et al. "Lynx: Enabling Efficient MoE Inference through Dynamic Batch-Aware Expert Selection." arXiv:2411.08982, 2024.
- Dengke Han. "DraftExpert: Expansion-Aware Self-Speculative Decoding for End-Device MoE Inference." arXiv:2607.24434, 2026.

- Awni Hannun, Jagrit Digani, Angelos Katharopoulos and Ronan Collobert. "MLX: Efficient and flexible machine learning on Apple silicon" (software, 2023; MIT). https://github.com/ml-explore/mlx
- Horace He and Thinking Machines Lab. "Defeating Nondeterminism in LLM Inference." Thinking Machines Lab: Connectionism, 10 Sep 2025. doi:10.64434/tml.20250910. https://thinkingmachines.ai/blog/defeating-nondeterminism-in-llm-inference/
- Dan Hendrycks et al. "Measuring Mathematical Problem Solving With the MATH Dataset." arXiv:2103.03874, 2021.
- Yuxuan Hu et al. "SAM Decoding: Speculative Decoding via Suffix Automaton." arXiv:2411.10666, 2024.
- Siming Huang. "Prefetch-Steered Routing: Deterministic Cache-Aware Expert Selection for Offloaded Mixture-of-Experts Decoding." arXiv:2610.34721, 2026.
- Wei Huang et al. "Mixture Compressor for Mixture-of-Experts LLMs Gains More." arXiv:2410.06270, 2024.
- Yuegui Huang et al. "DyMoE: Dynamic Expert Orchestration with Mixed-Precision Quantization for Efficient MoE Inference on Edge." arXiv:2603.19172, 2026.
- Ranggi Hwang et al. "Pre-gated MoE: An Algorithm-System Co-Design for Fast and Scalable Mixture-of-Expert Inference." arXiv:2308.12066, 2023.
- Inco AI. "DFlash 2: Keep Drafting Parallel." Blog, Aug 2026. https://inco.ai/blog/dflash2
- Inco AI. Qwen3.8-Flash-Next-DFlash2 (drafter checkpoint, revision a22a174, 19 Oct 2026; Apache-2.0). https://huggingface.co/incoai/Qwen3.8-Flash-Next-DFlash2
- Inco AI. Splash (software, release 1.4.0). https://github.com/incoai/splash
- ISTA-DASLab. Qwen3.8-Flash-Next-GSQ-RCO-GGUF (model files in GGUF format; Apache-2.0). Hugging Face. https://huggingface.co/ISTA-DASLab/Qwen3.8-Flash-Next-GSQ-RCO-GGUF
- Haoyun Jiang et al. "TriSpec: Ternary Speculative Decoding via Lightweight Proxy Verification." arXiv:2601.23180, 2026.
- Keisuke Kamahori et al. "Fiddler: CPU-GPU Orchestration for Fast Inference of Mixture-of-Experts Models." arXiv:2402.07033, 2024.
- Byeongju Kim et al. "FlashMoE: Reducing SSD I/O Bottlenecks via ML-Based Cache Replacement for Mixture-of-Experts Inference on Edge Devices." arXiv:2601.17063, 2026.
- Rui Kong et al. "SwapMoE: Serving Off-the-shelf MoE-based Large Language Models with Tunable Memory Budget." arXiv:2308.15030, 2023.
- Young D. Kwon et al. "WhiFlash: Accelerating Speculative Decoding with Token-Level Cross-Paradigm Routing." arXiv:2606.07710, 2026.
- Yaniv Leviathan, Matan Kalman and Yossi Matias. "Fast Inference from Transformers via Speculative Decoding." arXiv:2211.17192, 2022. ICML 2023.
- Yuhui Li et al. "EAGLE-3: Scaling up Inference Acceleration of Large Language Models via Training-Time Test." arXiv:2503.01840, 2025.
- Shuang Liang et al. "AcceptMoE: Commitment-Weighted Self-Sizing Verifier Expert Sets for Efficient MoE Speculative Decoding." arXiv:2608.02989, 2026.
- Hunter Lightman et al. "Let's Verify Step by Step." arXiv:2305.20050, 2023.
- Yutong Lin et al. "PagedMoE: Cross-Precision Expert Caching for Token-Level Dynamic Precision in SSD-Resident MoE Inference." arXiv:2611.03173, 2026.
- Jiawei Liu et al. "Is Your Code Generated by ChatGPT Really Correct? Rigorous Evaluation of Large Language Models for Code Generation." arXiv:2305.01210, 2023.
- Zechun Liu et al. "SpinQuant: LLM quantization with learned rotations." arXiv:2405.16406, 2024.
- Zhenyu Liu et al. "Bandwidth-Efficient Adaptive Mixture-of-Experts via Low-Rank Compensation." arXiv:2512.17073, 2025.
- LMSYS Org. "Towards Deterministic Inference in SGLang and Reproducible RL Training." LMSYS blog, 22 Sep 2025. https://lmsys.org/blog/2025-09-22-sglang-deterministic/
- Massimiliano Fasi, Nicholas J. Higham, Mantas Mikaitis and Srikara Pranesh. "Numerical behavior of NVIDIA tensor cores." PeerJ Computer Science 7:e330, 2021. doi:10.7717/peerj-cs.330
- Bradley McDanel et al. "MoE-Spec: Expert Budgeting for Efficient Speculative Decoding." arXiv:2602.16052, 2026.
- Quinn McNemar. "Note on the sampling error of the difference between correlated proportions or percentages." Psychometrika 12(2): 153-157, 1947. doi:10.1007/BF02295996.
- Mference (software). https://github.com/NeelM0906/Mference
- Microsoft. "NVMe" (power management for storage hardware devices: the StorNVMe driver's power states, entry and exit latencies, idle timeouts). Windows hardware design guidance, Microsoft Learn. https://learn.microsoft.com/en-us/windows-hardware/design/component-guidelines/power-management-for-storage-hardware-devices-nvme

- MLX issue #4632, "[BUG] gather_qmm(sorted_indices=True) returns wrong results on M5 Max (macOS 27) when rows > 32768 and not a multiple of 64", 5 Oct 2026. https://github.com/ml-explore/mlx/issues/4632
- MLX pull request #1773, "Faster synchronization `Fence` primitive", merged 18 Jan 2025. https://github.com/ml-explore/mlx/pull/1773
- MLX pull request #3922, "Fix sorted gather_qmm NAX row overflow above 32K", merged 26 Aug 2026. https://github.com/ml-explore/mlx/pull/3922
- mlx-lm (software; MIT). https://github.com/ml-explore/mlx-lm
- nace-ai. Kapitsa (software). https://github.com/nace-ai/kapitsa
- Pranav Nair et al. "Matryoshka Quantization." arXiv:2502.06786, 2025.
- Niko1221 and the Strata contributors. Strata (software, version 0.1.58-metal, commit 146df3c; MIT). https://github.com/Niko1221/Strata
- NVIDIA. Magnum IO GPUDirect Storage. https://developer.nvidia.com/gpudirect-storage
- Costin-Andrei Oncescu et al. "Opportunistic Expert Activation: Batch-Aware Expert Routing for Faster Decode Without Retraining." arXiv:2511.02237, 2025.
- ONNX Runtime (software; MIT). https://github.com/microsoft/onnxruntime
- Lehan Pan et al. "Making Every Verified Token Count: Adaptive Verification for MoE Speculative Decoding." arXiv:2605.00342, 2026.
- Zihan Qiu et al. "On the Design of Qwen3.8-Next Architecture: Evaluation, Efficiency, and Training Stability." arXiv:2608.30320, 2026.
- Zaid Qureshi et al. "GPU-Initiated On-Demand High-Throughput Storage Access in the BaM System Architecture." arXiv:2203.04910, 2022.
- Qwen Team. Qwen3.8-Flash-Next (model weights; Qwen Community License 1.0). https://huggingface.co/Qwen/Qwen3.8-Flash-Next
- Anish Saxena et al. "Utility-Driven Speculative Decoding for Mixture-of-Experts." arXiv:2506.20675, 2025.
- Apoorv Saxena. "Prompt Lookup Decoding" (software), Nov 2023. https://github.com/apoorvumang/prompt-lookup-decoding
- SK hynix. "Peripheral component interconnect express (PCIe) interface system and method of operating the same." US Patent 11,789,658 B2. https://patents.google.com/patent/US11789658B2/en
- Andrii Skliar et al. "Mixture of Cache-Conditional Experts for Efficient Mobile Device Inference." arXiv:2412.00099, 2024.
- SlotStream (software). https://github.com/carloslfu/slotstream. Issue #5, "Measured: Mac14,3, 16GB", 2 Sep 2026: https://github.com/carloslfu/slotstream/issues/5
- Jingwei Song et al. "MARS: Unleashing the Power of Speculative Decoding via Margin-Aware Verification." arXiv:2601.15498, 2026.
- Xiaoniu Song et al. "ProMoE: Fast MoE-based LLM Serving using Proactive Caching." arXiv:2410.22134, 2024.
- Lukas Stepanek. "Memory-Sovereign Inference: Output-Exact Execution Beyond Full Residency." arXiv:2608.23805, 2026.
- Sushi (software, release 1.4.0; MIT). https://github.com/beamivalice/sushi
- Peng Tang et al. "HOBBIT: A Mixed Precision Expert Offloading System for Fast MoE Inference." arXiv:2411.01433, 2024.
- Tencent Youtu Research. Palm-Infra (software; Apache-2.0), including docs/ssd-offload.md. https://github.com/TencentYoutuResearch/Palm-Infra
- Thinking Machines Lab. batch_invariant_ops (software; MIT). https://github.com/thinking-machines-lab/batch_invariant_ops
- Albert Tseng et al. "QTIP: Quantization with Trellises and Incoherence Processing." arXiv:2406.11235, 2024.
- Oszkár Urbán et al. "Margins, Not Windows: Training-Free Per-Step Lossy Speculative Decoding." arXiv:2609.02897, 2026.
- vLLM project. "Batch Invariance." vLLM documentation. https://docs.vllm.ai/en/latest/features/batch_invariance/
- Chiatzen Wang et al. "Acceptance Is Not Speed: Length Inflation in Training-Free Lossy Speculative Decoding." arXiv:2610.23818, 2026.
- Tianyu Wang et al. "Revisiting Lossy Verification in Speculative Decoding: Mechanisms, Trade-offs, and Failure Modes." arXiv:2607.26627, 2026.
- Wenfeng Wang et al. "Specter: Fast MoE Inference via Quantization-Enabled Multi-Step Speculative Offloading." arXiv:2511.14102, 2025.
- Ziyi Wang et al. "DIVERSED: Relaxed Speculative Decoding via Dynamic Ensemble Verification."

arXiv:2604.07622, 2026.
- Jincheng Xie et al. "Less Experts, Faster Decoding: Cost-Aware Speculative Decoding for Mixture-of-Experts." arXiv:2607.12696, 2026.
- Leyang Xue et al. "MoE-Infinity: Efficient MoE Inference on Personal Machines with Sparsity-Aware Expert Cache." arXiv:2401.14361, 2024.
- YorkshireDev. Project Torpid: a disk drive sleep prevention utility (software; GPL-3.0). https://github.com/YorkshireDev/Torpid
- Jiebin Zhang et al. "DFlare: Scaling Up Draft Capacity for Block Diffusion Speculative Decoding." arXiv:2606.02091, 2026.
- Shuzhang Zhong et al. "AdapMoE: Adaptive Sensitivity-based Expert Gating and Management for Efficient MoE Inference." arXiv:2408.10284, 2024.
- Jeffrey Zhou et al. "Instruction-Following Evaluation for Large Language Models." arXiv:2311.07911, 2023.
- Qi Zhu, Meng Zhu, Bo Wu, Xipeng Shen, Kai Shen and Zhiying Wang. "Software Engagement with Sleeping CPUs." In Proceedings of the 15th Workshop on Hot Topics in Operating Systems (HotOS XV), 2015.

Companion papers:

- [DRAFTER] Kamen Levi. Drafting for a 125B mixture-of-experts model that does not fit in RAM: training its own MTP head, one drafter per text type, and the levers that failed. 2026. A companion paper of this set.
- [EVAL] Kamen Levi. Certifying lossy decoding on the model's known benchmarks: same-scores gates for a 125B mixture-of-experts model on one 64 GB Mac. 2026. A companion paper of this set.
- [KERNELS] Kamen Levi. Batch-invariant kernels on Apple's M5 GPU and its matrix units. 2026. A companion paper of this set.
- [RELEASE] Kamen Levi. Running a 125B mixture-of-experts model on a 64 GB Mac: the engine package. 2026. A companion paper of this set.
- [SMALL-RAM] Kamen Levi. A 125B mixture-of-experts model in 32, 24 and 12 GB of memory on one Mac. 2026. A companion paper of this set.